

```
// File: client/Assets/Scripts/picmgr.cs
1  using cfg;
2  using System.Collections;
3  using System.Collections.Generic;
4  using UnityEngine;
5  using UnityEngine.UI;
6  using DG.Tweening;
7  using System;
8  using TMPro;
9
10 #if UNITY_EDITOR
11 using UnityEditor;
12 #endif
13
14 public class picmgr : MonoBehaviour
15 {
16     private const float CardAspect = 102f / 148f;
17     private const float PuzzleMargin = 50f;
18
19     public Texture2D pic;
20     public int width=3;
21     public int height=3;
22
23     internal static picmgr instance;
24     internal float carWid
25     {
26         get
27         {
28             return trans.rect.width / width;
29         }
30     }
31     internal float carHei
32     {
33         get
34         {
35             return trans.rect.height / height;
36         }
37     }
38
39     private void Awake()
40     {
41         instance = this;
42         ResizeChapterContent();
43     }
44     public void ResizeChapterContent()
45     {
46         ResizePuzzleContentInParent();
47         return;
48     }
49 #if false
```

```
50 // 使用 CanvasScaler 的 referenceResolution，确保在不同分辨率下一致
51 Canvas canvas = GetComponentInParent<Canvas>();
52 float refWidth, refHeight, scaleFactor = 1f;
53
54 if (canvas != null)
55 {
56     CanvasScaler scaler = canvas.GetComponent<CanvasScaler>();
57     if (scaler != null && scaler.uiScaleMode != CanvasScaler.ScaleMode.ConstantPixelSize)
58     {
59         refWidth = scaler.referenceResolution.x;
60         refHeight = scaler.referenceResolution.y;
61         // 根据当前屏幕和 referenceResolution 计算缩放比例
62         scaleFactor = (float)Screen.height / refHeight;
63     }
64     else
65     {
66         // 没有 CanvasScaler 或 ConstantPixelSize，直接使用屏幕尺寸
67         refWidth = Screen.width;
68         refHeight = Screen.height;
69     }
70 }
71 else
72 {
73     refWidth = Screen.width;
74     refHeight = Screen.height;
75 }
76
77 // 边距根据缩放比例自适应
78 float margin = PuzzleMargin * scaleFactor;
79
80 // 计算可用空间
81 float availableWidth = refWidth - 2 * margin;
82 float availableHeight = refHeight - 2 * margin;
83
84 // 9:16 宽高比的目标宽高
85 float puzzleAspect = GetPuzzleAspect();
86 float targetWidth, targetHeight;
87
88 if (availableWidth / availableHeight > puzzleAspect)
89 {
90     // 以高度为基准
91     targetHeight = availableHeight;
92     targetWidth = targetHeight * puzzleAspect;
93 }
94 else
95 {
96     // 以宽度为基准
97     targetWidth = availableWidth;
98     targetHeight = targetWidth / puzzleAspect;
99 }
```

```
100
101     float wside = (refWidth - targetWidth) / 2;
102     float hside = (refHeight - targetHeight) / 2;
103     trans.offsetMin = new Vector2(wside, hside);
104     trans.offsetMax = new Vector2(-wside, -hside);
105 #endif
106 }
107
108 private void ResizePuzzleContentInParent()
109 {
110     Canvas.ForceUpdateCanvases();
111
112     RectTransform parent = trans.parent as RectTransform;
113     Vector2 parentSize = parent != null
114         ? parent.rect.size
115         : new Vector2(Screen.width, Screen.height);
116
117     float availableWidth = Mathf.Max(1f, parentSize.x - 2f * PuzzleMargin);
118     float availableHeight = Mathf.Max(1f, parentSize.y - 2f * PuzzleMargin);
119     float puzzleAspect = GetPuzzleAspect();
120
121     float targetWidth;
122     float targetHeight;
123     if (availableWidth / availableHeight > puzzleAspect)
124     {
125         targetHeight = availableHeight;
126         targetWidth = targetHeight * puzzleAspect;
127     }
128     else
129     {
130         targetWidth = availableWidth;
131         targetHeight = targetWidth / puzzleAspect;
132     }
133
134     trans.anchorMin = new Vector2(0.5f, 0.5f);
135     trans.anchorMax = new Vector2(0.5f, 0.5f);
136     trans.pivot = new Vector2(0.5f, 0.5f);
137     trans.anchoredPosition = Vector2.zero;
138     trans.sizeDelta = new Vector2(targetWidth, targetHeight);
139 }
140
141 private float GetPuzzleAspect()
142 {
143     if (width > 0 && height > 0)
144     {
145         return Mathf.Max(0.01f, CardAspect * width / height);
146     }
147
148     return CardAspect;
149 }
```

```
150
151 RectTransform trans
152 {
153     get
154     {
155         return GetComponent<RectTransform>();
156     }
157 }
158 DrLevel curlevel;
159 // 发牌动画是否完成，未完成时禁止拖拽
160 public bool cardsReady = false;
161 public IEnumerator LoadLevel(DrLevel leevel)
162 {
163     curlevel = leevel;
164     width = leevel.LevelFigureX;
165     height = leevel.LevelFigureY;
166
167     yield return Main.inst.StartCoroutine(Main.LoadTextureFromCDN(leevel.LevelFigure, (texture) =>
168     {
169         pic = texture;
170     }));
171     ResizeChapterContent();
172     Canvas.ForceUpdateCanvases();
173     Main.DispEvent("event_loading", false);
174
175     if(leevel.DifficultyTier==2)
176     {
177         Main.DispEvent("event_tips", "困难模式");
178     }
179
180     // 传递关卡ID、最大原位数量和难度参数
181     yield return Main.inst.StartCoroutine( CreateGridImages(leevel.Id, leevel.OutOfPlaceNumber, leevel.DifficultyTier == 2));
182 }
183 // Start is called before the first frame update
184
185 public void tCreateGridImages(int level = 1, int maxKeepCount = -1, bool isHard = false)
186 {
187     // 编辑器测试方法，使用默认参数
188     //StartCoroutine(CreateGridImages(1, -1, false));
189     clearOld();
190     ResizePuzzleContentInParent();
191     RectTransform rectTransform = GetComponent<RectTransform>();
192     Vector2 rectSize = rectTransform.rect.size;
193
194     // 计算每个格子的尺寸
195     float cellWidth = rectSize.x / width;
196     float cellHeight = rectSize.y / height;
197     float time_fp = 1f;
198     float delay = time_fp / (width * height);
199     // 创建width*height个格子
```

```
200     for (int x = 0; x < width; x++)
201     {
202         for (int y = 0; y < height; y++)
203         {
204             create(x, y, cellWidth, cellHeight, isHard, true);
205         }
206     }
207     ShuffleGridPositions(level, maxKeepCount, isHard);
208 }
209 // 添加的方法：根据当前recttransform尺寸拆分成width*height个格子，每个格子填充一个RawImage
210 // level: 关卡编号，用于确保每个关卡每次刷新出的位置是相同的
211 // maxKeepCount: 最多留在原位的卡牌数量，-1表示使用默认值
212 // isHard: 是否为困难模式
213 public IEnumerator CreateGridImages(int level = 1, int maxKeepCount = -1, bool isHard = false)
214 {
215     cardsReady = false;
216     clearOld();
217     ResizePuzzleContentInParent();
218
219     RectTransform rectTransform = GetComponent<RectTransform>();
220     Vector2 rectSize = rectTransform.rect.size;
221
222     // 计算每个格子的尺寸
223     float cellWidth = rectSize.x / width;
224     float cellHeight = rectSize.y / height;
225     float time_fp = 1f;
226     float delay = time_fp / (width * height);
227     // 创建width*height个格子
228     for (int x = 0; x < width; x++)
229     {
230         for (int y = 0; y < height; y++)
231         {
232             yield return new WaitForSeconds(delay);
233             create(x, y, cellWidth, cellHeight, isHard, false);
234         }
235     }
236     yield return new WaitForSeconds(0.3f);
237     ShuffleGridPositions(level, maxKeepCount, isHard);
238     for (int i = 0; i < transform.childCount; i++)
239     {
240         var x = transform.GetChild(i);
241         var d = x.GetComponent<DraggableGridItem>();
242         d.Turn();
243     }
244     // 等待所有 Turn 动画完成（并行执行，约 1 秒）后再允许拖拽
245     yield return new WaitForSeconds(1.1f);
246     cardsReady = true;
247 }
248
249 private void clearOld()
```

```
250 {
251     // 清除现有的子对象 (使用倒序删除避免遍历时修改集合的问题)
252     for (int i = transform.childCount - 1; i >= 0; i--)
253     {
254         Transform child = transform.GetChild(i);
255         // 在编辑器中使用DestroyImmediate, 在运行时使用Destroy
256         #if UNITY_EDITOR
257             if (Application.isPlaying)
258             {
259                 Destroy(child.gameObject);
260             }
261             else
262             {
263                 DestroyImmediate(child.gameObject);
264             }
265         #else
266             Destroy(child.gameObject);
267         #endif
268     }
269 }
270
271 private void create(int x, int y, float cellWidth, float cellHeight, bool isHard, bool beditor)
272 {
273     // 创建一个空的游戏对象
274     GameObject cellObject = GameObject.Instantiate(Resources.Load("GridCell") as GameObject);
275     cellObject.name = $"GridCell_{x}_{y}";
276     cellObject.transform.SetParent(transform, false);
277
278     // 添加RectTransform组件
279     RectTransform cellRect = cellObject.GetComponent<RectTransform>();
280
281     if (isHard)
282     {
283         cellObject.GetComponent<RawImage>().texture = (Texture) Resources.Load("Card/ui_card_02");// Color.red;
284     }
285     else
286     {
287         cellObject.GetComponent<RawImage>().texture = (Texture)Resources.Load("Card/ui_card_01");// Color.red;
288     }
289     // 添加拖拽组件
290     var dg = cellObject.GetComponent<DraggableGridItem>();
291     dg.pic = pic;
292     // 初始化时计算位置索引
293
294     // 计算并设置UV坐标
295     float uvX = (float)x / width;
296     float uvY = (float)y / height;
297     float uvWidth = 1.0f / width;
298     float uvHeight = 1.0f / height;
299 }
```

```
300
301     dg.canvas = GetComponentInParent<Canvas>();
302
303     dg.uvX = uvX;
304     dg.uvY = uvY;
305     dg.uvWidth = uvWidth;
306     dg.uvHeight = uvHeight;
307
308     dg.PositionIndex = x * height + y;
309     // 设置位置和尺寸
310     cellRect.anchorMin = new Vector2(0, 0);
311     cellRect.anchorMax = new Vector2(0, 0);
312     cellRect.pivot = new Vector2(0, 0);
313     if (beditor)
314     {
315         cellRect.anchoredPosition = new Vector2(x * cellWidth, y * cellHeight);
316         cellRect.GetComponent<DraggableGridItem>().Turn(true);
317
318
319     }
320     else
321     {
322         cellRect.anchoredPosition = new Vector2(0f, 0);
323
324         // 非编辑器模式下, 根据调试模式决定是否移除文本组件
325         bool isDebugMode = DebugManager.IsDebugMode;
326
327         if (!isDebugMode)
328         {
329             TextMeshProUGUI existingText = cellObject.GetComponentInChildren<TextMeshProUGUI>();
330             if (existingText != null)
331             {
332                 if (Application.isPlaying)
333                 {
334                     GameObject.Destroy(existingText.gameObject);
335                 }
336                 else
337                 {
338                     GameObject.DestroyImmediate(existingText.gameObject);
339                 }
340             }
341         }
342     }
343     cellRect.sizeDelta = new Vector2(cellWidth, cellHeight);
344
345     // 创建边框
346     CreateBorders(cellObject, cellWidth, cellHeight);
347
348     // 检查调试模式状态
349     bool isDebugModeForText = beditor || DebugManager.IsDebugMode;
```

```
350
351     if (isDebugModeForText)
352     {
353         // 调试模式下，添加文本显示位置信息
354         TextMeshProUGUI originalPosText = cellObject.GetComponentInChildren<TextMeshProUGUI>();
355         if (originalPosText == null)
356         {
357             // 创建TextMeshProUGUI组件
358             GameObject textObject = new GameObject("DebugText");
359             textObject.transform.SetParent(cellObject.transform, false);
360             originalPosText = textObject.AddComponent<TextMeshProUGUI>();
361
362             // 设置文本组件的属性
363             RectTransform textRect = textObject.GetComponent<RectTransform>();
364             textRect.anchorMin = new Vector2(0.5f, 0.5f);
365             textRect.anchorMax = new Vector2(0.5f, 0.5f);
366             textRect.pivot = new Vector2(0.5f, 0.5f);
367             textRect.anchoredPosition = Vector2.zero;
368             textRect.sizeDelta = new Vector2(cellWidth, cellHeight);
369
370             // 设置文本样式
371             originalPosText.fontSize = 24;
372             originalPosText.alignment = TextAlignmentOptions.Center;
373             originalPosText.color = Color.red;
374             originalPosText.fontStyle = FontStyles.Bold;
375             originalPosText.outlineWidth = 0.2f;
376             originalPosText.outlineColor = Color.black;
377
378             // 设置文本层级，确保显示在卡牌上方
379             Canvas canvas = cellObject.GetComponentInParent<Canvas>();
380             if (canvas != null)
381             {
382                 textObject.layer = canvas.gameObject.layer;
383             }
384         }
385
386         // 更新文本内容，显示网格坐标和位置索引
387         var xdg = cellObject.GetComponent<DraggableGridItem>();
388         originalPosText.text = "";// $"({x}, {y})\nIndex: {xdg.PositionIndex}";
389     }
390
391
392     var pos = new Vector2(x * cellWidth, y * cellHeight);
393     cellRect.DOAnchorPos(pos, 0.2f).onComplete = () =>
394     {
395         cellRect.anchoredPosition = pos;
396     };
397
398 }
399
```



```
400 // 创建边框
401 private void CreateBorders(GameObject cellObject, float cellWidth, float cellHeight)
402 {
403     for(int i = cellObject.transform.childCount - 1; i >= 0; i--)
404     {
405         if (Application.isPlaying)
406         {
407             GameObject.Destroy(cellObject.transform.GetChild(i).gameObject);
408         }
409         else
410         {
411             GameObject.DestroyImmediate(cellObject.transform.GetChild(i).gameObject);
412         }
413     }
414
415     // 边框宽度（用于边以及角的边长）
416     float borderWidth = 20f;
417
418     // 为防止角与边重合，将角大小限制为单元尺寸的一半以内
419     float bw = Mathf.Min(borderWidth, cellWidth / 2f, cellHeight / 2f);
420
421     // 创建四个角（左上、右上、右下、左下）——使用正方形区域
422     CreateBorder(cellObject, "TopLeftBorder", new Vector2(0, cellHeight - bw), new Vector2(bw, bw));
423     CreateBorder(cellObject, "TopRightBorder", new Vector2(cellWidth - bw, cellHeight - bw), new Vector2(bw, bw));
424     CreateBorder(cellObject, "BottomRightBorder", new Vector2(cellWidth - bw, 0), new Vector2(bw, bw));
425     CreateBorder(cellObject, "BottomLeftBorder", new Vector2(0, 0), new Vector2(bw, bw));
426
427     // 创建四个边（上、下、左、右），缩进到角的内部以避免重合
428     // 上/下：从 x = bw 开始，宽度为 cellWidth - 2*bw
429     CreateBorder(cellObject, "TopBorder", new Vector2(bw, cellHeight - bw), new Vector2(Mathf.Max(0f, cellWidth - 2f * bw), bw));
430     CreateBorder(cellObject, "BottomBorder", new Vector2(bw, 0), new Vector2(Mathf.Max(0f, cellWidth - 2f * bw), bw));
431
432     // 左/右：从 y = bw 开始，高度为 cellHeight - 2*bw
433     CreateBorder(cellObject, "LeftBorder", new Vector2(0, bw), new Vector2(bw, Mathf.Max(0f, cellHeight - 2f * bw)));
434     CreateBorder(cellObject, "RightBorder", new Vector2(cellWidth - bw, bw), new Vector2(bw, Mathf.Max(0f, cellHeight - 2f * bw)));
435 }
436
437 // 创建单个边框
438 private void CreateBorder(GameObject parent, string name, Vector2 anchoredPosition, Vector2 sizeDelta)
439 {
440     GameObject borderObject = new GameObject(name);
441     borderObject.transform.SetParent(parent.transform, false);
442
443     RectTransform borderRect = borderObject.AddComponent<RectTransform>();
444     borderRect.anchorMin = new Vector2(0, 0);
445     borderRect.anchorMax = new Vector2(0, 0);
446     borderRect.pivot = new Vector2(0, 0);
447     borderRect.anchoredPosition = anchoredPosition;
448     borderRect.sizeDelta = sizeDelta;
449 }
```

```
450     Image borderImage = borderObject.AddComponent<Image>();
451     borderImage.color = Color.white; // 边框颜色设为红色以便观察
452
453     // 为边框添加一个加载器组件，默认从 Resources 目录根据约定名称加载贴图
454     BorderSpriteLoader loader = borderObject.AddComponent<BorderSpriteLoader>();
455     loader.resourceName = MapBorderNameToResource(name);
456     loader.LoadSprite();
457 }
458
459 // 将边框对象名映射到 Resources 中的贴图名（约定）
460 private string MapBorderNameToResource(string borderName)
461 {
462     switch (borderName)
463     {
464         case "TopLeftBorder": return "lt";
465         case "TopBorder": return "t";
466         case "TopRightBorder": return "rt";
467         case "RightBorder": return "r";
468         case "BottomRightBorder": return "rb";
469         case "BottomBorder": return "b";
470         case "BottomLeftBorder": return "lb";
471         case "LeftBorder": return "l";
472         default: return borderName; // fallback: 使用相同名字
473     }
474 }
475
476 // 更新边框显示状态（非拖拽项）
477 public void UpdateBorderVisibility()
478 {
479     List<DraggableGridItem> allItems = new List<DraggableGridItem>();
480     foreach (Transform child in transform)
481     {
482         DraggableGridItem item = child.GetComponent<DraggableGridItem>();
483         if (item != null && !item.IsBeingDragged)
484             allItems.Add(item);
485     }
486     if (allItems.Count > 0)
487         UpdateBorderVisibility(allItems);
488 }
489
490 // 更新边框显示状态（指定组，拖拽项专用）
491 public void UpdateBorderVisibility(List<DraggableGridItem> group)
492 {
493     if (group == null || group.Count == 0) return;
494
495     // 计算每个格子的尺寸
496     RectTransform rectTransform = GetComponent<RectTransform>();
497     Vector2 rectSize = rectTransform.rect.size;
498     float cellWidth = rectSize.x / width;
499     float cellHeight = rectSize.y / height;
```

```
500
501 // 检查每对相邻节点的关系
502 foreach (DraggableGridItem item in group)
503 {
504     RectTransform itemRect = item.GetComponent<RectTransform>();
505     if (itemRect == null) continue;
506
507     // 解析节点坐标 (逻辑位置)
508     string[] parts = item.gameObject.name.Replace("GridCell_", "").Split('_');
509     if (parts.Length != 2) continue;
510
511     int x, y;
512     if (!int.TryParse(parts[0], out x) || !int.TryParse(parts[1], out y)) continue;
513
514     // 检查四个方向的逻辑邻居
515     float checkThreshold = 1.5f;
516
517     Vector2 expectedRightPos = itemRect.anchoredPosition + new Vector2(cellWidth, 0);
518     Vector2 expectedLeftPos = itemRect.anchoredPosition + new Vector2(-cellWidth, 0);
519     Vector2 expectedTopPos = itemRect.anchoredPosition + new Vector2(0, cellHeight);
520     Vector2 expectedBottomPos = itemRect.anchoredPosition + new Vector2(0, -cellHeight);
521
522     DraggableGridItem rightNeighborItem = FindGridItemAtAnchoredPosition(expectedRightPos, group, checkThreshold);
523     DraggableGridItem leftNeighborItem = FindGridItemAtAnchoredPosition(expectedLeftPos, group, checkThreshold);
524     DraggableGridItem topNeighborItem = FindGridItemAtAnchoredPosition(expectedTopPos, group, checkThreshold);
525     DraggableGridItem bottomNeighborItem = FindGridItemAtAnchoredPosition(expectedBottomPos, group, checkThreshold);
526
527     Vector2 expectedTopRightPos = itemRect.anchoredPosition + new Vector2(cellWidth, cellHeight);
528     Vector2 expectedTopLeftPos = itemRect.anchoredPosition + new Vector2(-cellWidth, cellHeight);
529     Vector2 expectedBottomRightPos = itemRect.anchoredPosition + new Vector2(cellWidth, -cellHeight);
530     Vector2 expectedBottomLeftPos = itemRect.anchoredPosition + new Vector2(-cellWidth, -cellHeight);
531
532     DraggableGridItem topRightNeighborItem = FindGridItemAtAnchoredPosition(expectedTopRightPos, group, checkThreshold);
533     DraggableGridItem topLeftNeighborItem = FindGridItemAtAnchoredPosition(expectedTopLeftPos, group, checkThreshold);
534     DraggableGridItem bottomRightNeighborItem = FindGridItemAtAnchoredPosition(expectedBottomRightPos, group, checkThr ...
535     DraggableGridItem bottomLeftNeighborItem = FindGridItemAtAnchoredPosition(expectedBottomLeftPos, group, checkThres ...
536
537     bool isRightCorrect = (rightNeighborItem != null && rightNeighborItem.gameObject.name == ("GridCell_" + (x + 1) + "_" + y));
538     bool isLeftCorrect = (leftNeighborItem != null && leftNeighborItem.gameObject.name == ("GridCell_" + (x - 1) + "_" + y));
539     bool isTopCorrect = (topNeighborItem != null && topNeighborItem.gameObject.name == ("GridCell_" + x + "_" + (y + 1)));
540     bool isBottomCorrect = (bottomNeighborItem != null && bottomNeighborItem.gameObject.name == ("GridCell_" + x + "_" + (y - ...
541
542     bool isTopRightCorrect = (topRightNeighborItem != null && topRightNeighborItem.gameObject.name == ("GridCell_" + (x + 1) + ...
543     bool isTopLeftCorrect = (topLeftNeighborItem != null && topLeftNeighborItem.gameObject.name == ("GridCell_" + (x - 1) + "_" + ...
544     bool isBottomRightCorrect = (bottomRightNeighborItem != null && bottomRightNeighborItem.gameObject.name == ("GridCell ...
545     bool isBottomLeftCorrect = (bottomLeftNeighborItem != null && bottomLeftNeighborItem.gameObject.name == ("GridCell_" + ...
546
547     bool rightShown = !isRightCorrect;
548     bool leftShown = !isLeftCorrect;
549     bool topShown = !isTopCorrect;
```

```
550         bool bottomShown = !isBottomCorrect;
551
552         item.adjacentRight = isRightCorrect;
553         item.adjacentLeft = isLeftCorrect;
554         item.adjacentTop = isTopCorrect;
555         item.adjacentBottom = isBottomCorrect;
556
557         GameObject rightBorder = FindChildByName(item.gameObject, "RightBorder");
558         GameObject leftBorder = FindChildByName(item.gameObject, "LeftBorder");
559         GameObject topBorder = FindChildByName(item.gameObject, "TopBorder");
560         GameObject bottomBorder = FindChildByName(item.gameObject, "BottomBorder");
561
562         GameObject topLeftBorder = FindChildByName(item.gameObject, "TopLeftBorder");
563         GameObject topRightBorder = FindChildByName(item.gameObject, "TopRightBorder");
564         GameObject bottomRightBorder = FindChildByName(item.gameObject, "BottomRightBorder");
565         GameObject bottomLeftBorder = FindChildByName(item.gameObject, "BottomLeftBorder");
566
567         if (rightBorder != null)
568             rightBorder.SetActive(rightShown);
569
570         if (leftBorder != null)
571             leftBorder.SetActive(leftShown);
572
573         if (topBorder != null)
574             topBorder.SetActive(topShown);
575
576         if (bottomBorder != null)
577             bottomBorder.SetActive(bottomShown);
578
579         if (topLeftBorder != null)
580             topLeftBorder.SetActive(topShown || leftShown || !isTopLeftCorrect);
581
582         if (topRightBorder != null)
583             topRightBorder.SetActive(topShown || rightShown || !isTopRightCorrect);
584
585         if (bottomRightBorder != null)
586             bottomRightBorder.SetActive(bottomShown || rightShown || !isBottomRightCorrect);
587
588         if (bottomLeftBorder != null)
589             bottomLeftBorder.SetActive(bottomShown || leftShown || !isBottomLeftCorrect);
590
591         if (topLeftBorder != null)
592         {
593             BorderSpriteLoader bl = topLeftBorder.GetComponent<BorderSpriteLoader>();
594             if (bl != null) bl.RefreshBasedOnFlags("lt", isTopCorrect, isLeftCorrect, isRightCorrect, isBottomCorrect, isTopLeftCorrect);
595         }
596
597         if (topRightBorder != null)
598         {
599             BorderSpriteLoader bl = topRightBorder.GetComponent<BorderSpriteLoader>();
```

```

600         if (bl != null) bl.RefreshBasedOnFlags("rt", isTopCorrect, isLeftCorrect, isRightCorrect, isBottomCorrect, isTopRightCorrect);
601     }
602
603     if (bottomRightBorder != null)
604     {
605         BorderSpriteLoader bl = bottomRightBorder.GetComponent<BorderSpriteLoader>();
606         if (bl != null) bl.RefreshBasedOnFlags("rb", isTopCorrect, isLeftCorrect, isRightCorrect, isBottomCorrect, isBottomRightCorrect);
607     }
608
609     if (bottomLeftBorder != null)
610     {
611         BorderSpriteLoader bl = bottomLeftBorder.GetComponent<BorderSpriteLoader>();
612         if (bl != null) bl.RefreshBasedOnFlags("lb", isTopCorrect, isLeftCorrect, isRightCorrect, isBottomCorrect, isBottomLeftCorrect);
613     }
614
615     // 额外规则：如果当前格子的左右都是正确邻居，则把左侧节点的右下角和右侧节点的左下角设为底部贴图("b")
616     //if (isLeftCorrect && isRightCorrect)
617     //{
618     //    if (leftNeighborItem != null)
619     //    {
620     //        GameObject neighBR = FindChildByName(leftNeighborItem.gameObject, "BottomRightBorder");
621     //        if (neighBR != null)
622     //        {
623     //            BorderSpriteLoader bl = neighBR.GetComponent<BorderSpriteLoader>();
624     //            if (bl != null)
625     //            {
626     //                // 如果左侧邻居是网格角（任一角），跳过覆盖
627     //                string[] lnParts = leftNeighborItem.gameObject.name.Replace("GridCell_", "").Split('_');
628     //                int lnx = int.Parse(lnParts[0]);
629     //                int lny = int.Parse(lnParts[1]);
630     //                bool lnIsCorner = (lnx == 0 && lny == (height - 1)) || (lnx == (width - 1) && lny == (height - 1)) || (lnx == (width - 1) && lny = ...
631     //                if (!lnIsCorner) bl.SetResourceAndLoad("b");
632     //            }
633     //        }
634     //    }
635     //    if (rightNeighborItem != null)
636     //    {
637     //        GameObject neighBL = FindChildByName(rightNeighborItem.gameObject, "BottomLeftBorder");
638     //        if (neighBL != null)
639     //        {
640     //            BorderSpriteLoader bl = neighBL.GetComponent<BorderSpriteLoader>();
641     //            if (bl != null)
642     //            {
643     //                // 如果右侧邻居是网格角（任一角），跳过覆盖
644     //                string[] rnParts = rightNeighborItem.gameObject.name.Replace("GridCell_", "").Split('_');
645     //                int rnx = int.Parse(rnParts[0]);
646     //                int rny = int.Parse(rnParts[1]);
647     //                bool rnIsCorner = (rnx == 0 && rny == (height - 1)) || (rnx == (width - 1) && rny == (height - 1)) || (rnx == (width - 1) && rny ...
648     //                if (!rnIsCorner) bl.SetResourceAndLoad("b");
649     //            }
650     //        }
651     //    }

```

```
650     // }
651     // }
652     //}
653 }
654 }
655 /*
656  * 检查胜利
657 */
658 internal void CheckSucess()
659 {
660     bool suc = true;
661     foreach (Transform child in transform)
662     {
663         var c = child.GetComponent<DraggableGridItem>();
664         if (c == null) continue;
665
666         // 解析当前卡牌名称，格式为 "GridCell_x_y"
667         string[] parts = child.name.Replace("GridCell_", "").Split('_');
668         if (parts.Length != 2) continue;
669
670         int x, y;
671         if (!int.TryParse(parts[0], out x) || !int.TryParse(parts[1], out y)) continue;
672
673         // 计算预期的邻居名称
674         string expectedRightName = "GridCell_" + (x + 1) + "_" + y;
675         string expectedLeftName = "GridCell_" + (x - 1) + "_" + y;
676         string expectedTopName = "GridCell_" + x + "_" + (y + 1);
677         string expectedBottomName = "GridCell_" + x + "_" + (y - 1);
678
679         // 查找实际的邻居
680         DraggableGridItem rightNeighbor = null;
681         DraggableGridItem leftNeighbor = null;
682         DraggableGridItem topNeighbor = null;
683         DraggableGridItem bottomNeighbor = null;
684
685         foreach (Transform otherChild in transform)
686         {
687             if (otherChild == child) continue;
688
689             var otherItem = otherChild.GetComponent<DraggableGridItem>();
690             if (otherItem == null) continue;
691
692             // 检查相邻位置
693             RectTransform otherRect = otherChild.GetComponent<RectTransform>();
694             RectTransform currentRect = child.GetComponent<RectTransform>();
695             if (otherRect == null || currentRect == null) continue;
696
697             // 计算位置差异
698             Vector2 diff = otherRect.anchoredPosition - currentRect.anchoredPosition;
699             float threshold = 1.0f; // 位置差异阈值
```

```
700
701     if (Mathf.Abs(diff.x - currentRect.sizeDelta.x) < threshold && Mathf.Abs(diff.y) < threshold)
702     {
703         rightNeighbor = otherItem;
704     }
705     else if (Mathf.Abs(diff.x + currentRect.sizeDelta.x) < threshold && Mathf.Abs(diff.y) < threshold)
706     {
707         leftNeighbor = otherItem;
708     }
709     else if (Mathf.Abs(diff.y - currentRect.sizeDelta.y) < threshold && Mathf.Abs(diff.x) < threshold)
710     {
711         topNeighbor = otherItem;
712     }
713     else if (Mathf.Abs(diff.y + currentRect.sizeDelta.y) < threshold && Mathf.Abs(diff.x) < threshold)
714     {
715         bottomNeighbor = otherItem;
716     }
717 }
718
719 // 检查邻居名称是否正确
720 bool isRightCorrect = (rightNeighbor == null) || (rightNeighbor.gameObject.name == expectedRightName);
721 bool isLeftCorrect = (leftNeighbor == null) || (leftNeighbor.gameObject.name == expectedLeftName);
722 bool isTopCorrect = (topNeighbor == null) || (topNeighbor.gameObject.name == expectedTopName);
723 bool isBottomCorrect = (bottomNeighbor == null) || (bottomNeighbor.gameObject.name == expectedBottomName);
724
725 // 如果任何一个方向不正确, 设置suc为false
726 if (!isRightCorrect || !isLeftCorrect || !isTopCorrect || !isBottomCorrect)
727 {
728     suc = false;
729     break;
730 }
731 }
732
733 if (suc)
734 {
735     var t = transform.GetComponent<RectTransform>();
736     if (t != null)
737     {
738         // 向上移动100像素
739         Vector2 currentPos = t.anchoredPosition;
740         t.DOAnchorPos(new Vector2(currentPos.x, currentPos.y + 100f), 0.5f).onComplete=() =>
741         {
742             // 显示胜利界面
743             for(int i = 0;i< transform.childCount; i++)
744             {
745                 var x= transform.GetChild(i);
746                 x.GetComponent<DraggableGridItem>().enabled=false;
747             }
748
749             Main.DispEvent("show_victory", curlevel);
```

```
750         clearOld();
751     };
752 }
753 }
754 }
755
756
757 // 根据名称查找网格项
758 private DraggableGridItem FindGridItemByName(string name)
759 {
760     foreach (Transform child in transform)
761     {
762         if (child.name == name)
763         {
764             return child.GetComponent<DraggableGridItem>();
765         }
766     }
767     return null;
768 }
769
770 // 在指定的anchoredPosition附近查找网格项（根据实际位置检测）
771 private DraggableGridItem FindGridItemAtAnchoredPosition(Vector2 anchoredPosition, List<DraggableGridItem> gridItems, float t ...
772 {
773     foreach (DraggableGridItem item in gridItems)
774     {
775         RectTransform rt = item.GetComponent<RectTransform>();
776         if (rt == null) continue;
777         if (Vector2.Distance(rt.anchoredPosition, anchoredPosition) <= threshold)
778         {
779             return item;
780         }
781     }
782     return null;
783 }
784
785 // 根据名称查找子对象
786 private GameObject FindChildByName(GameObject parent, string name)
787 {
788     Transform child = parent.transform.Find(name);
789     return child != null ? child.gameObject : null;
790 }
791
792 // 刷新所有卡牌的位置索引
793 public void RefreshAllPositionIndices()
794 {
795     foreach (Transform child in transform)
796     {
797         DraggableGridItem item = child.GetComponent<DraggableGridItem>();
798         if (item != null)
799         {
```



```
800         // 根据当前位置更新位置索引
801         int x = Mathf.RoundToInt(item.rectTransform.anchoredPosition.x / carWid);
802         int y = Mathf.RoundToInt(item.rectTransform.anchoredPosition.y / carHei);
803         item.PositionIndex = x * this.height + y;
804         var originalPosText = item.GetComponentInChildren<TextMeshProUGUI>();
805         if(originalPosText != null)
806         {
807             originalPosText.text = item.PositionIndex.ToString();
808         }
809     }
810 }
811 }
812
813 // 新增方法：计算两点之间的曼哈顿距离（用于网格距离计算）
814 private int CalculateManhattanDistance(int x1, int y1, int x2, int y2)
815 {
816     return Mathf.Abs(x1 - x2) + Mathf.Abs(y1 - y2);
817 }
818
819 // 修改后的ShuffleGridPositions方法
820 public void ShuffleGridPositions(int level = 1, int maxKeepCount = -1, bool isHard = false)
821 {
822     // 收集所有子对象
823     List<RectTransform> children = new List<RectTransform>();
824     List<Vector2> originalPositions = new List<Vector2>();
825     List<Vector2Int> originalGridCoords = new List<Vector2Int>(); // 存储原始网格坐标
826
827     foreach (Transform child in transform)
828     {
829         RectTransform rect = child.GetComponent<RectTransform>();
830         if (rect != null)
831         {
832             children.Add(rect);
833             originalPositions.Add(rect.anchoredPosition);
834
835             // 解析原始网格坐标
836             string[] parts = child.name.Replace("GridCell_", "").Split('_');
837             if (parts.Length == 2)
838             {
839                 int x, y;
840                 if (int.TryParse(parts[0], out x) && int.TryParse(parts[1], out y))
841                 {
842                     originalGridCoords.Add(new Vector2Int(x, y));
843                     continue;
844                 }
845             }
846             originalGridCoords.Add(new Vector2Int(-1, -1)); // 解析失败的情况
847         }
848     }
849 }
```

```
850     // 如果子对象数量不足2个，无法打乱
851     if (children.Count < 2)
852     {
853         return;
854     }
855
856     // 获取网格单元的尺寸（从第一个子对象获取）
857     Vector2 cellSize = children[0].sizeDelta;
858
859     // 创建可用位置列表（初始为所有位置）
860     List<int> availableIndices = new List<int>();
861     for (int i = 0; i < children.Count; i++)
862     {
863         availableIndices.Add(i);
864     }
865
866     // 计算网格的宽高
867     int gridWidth = width;
868     int gridHeight = height;
869
870     // 计算默认maxKeepCount：总卡牌数的30%
871     if (maxKeepCount == -1)
872     {
873         maxKeepCount = Mathf.Max(0, Mathf.RoundToInt(children.Count * 0.3f));
874     }
875     // 确保maxKeepCount在有效范围内
876     maxKeepCount = Mathf.Clamp(maxKeepCount, 0, children.Count - 1);
877
878     // 随机选择要保留在原位的卡牌数量（0到maxKeepCount之间）
879     // 使用关卡编号和难度作为随机种子，确保每次结果一致
880     int seed = level * 1000 + (isHard ? 1 : 0);
881     System.Random rng = new System.Random(seed);
882     int cardsToKeep = rng.Next(0, maxKeepCount + 1);
883
884     // 随机选择要保留在原位的卡牌索引
885     List<int> keepIndices = new List<int>();
886     List<int> allIndices = new List<int>();
887     for (int i = 0; i < children.Count; i++)
888     {
889         allIndices.Add(i);
890     }
891
892     // 打乱allIndices列表，使用固定种子确保结果一致
893     System.Random shuffleRng = new System.Random(seed + 1);
894     for (int i = allIndices.Count - 1; i > 0; i--)
895     {
896         int j = shuffleRng.Next(0, i + 1);
897         int temp = allIndices[i];
898         allIndices[i] = allIndices[j];
899         allIndices[j] = temp;
```

```
900     }
901
902     // 选择前cardsToKeep个索引作为要保留的卡牌
903     for (int i = 0; i < cardsToKeep; i++)
904     {
905         keepIndices.Add(allIndices[i]);
906         // 从可用位置列表中移除这些索引，它们将保持原位
907         availableIndices.Remove(allIndices[i]);
908     }
909
910     // 为剩余的卡牌分配新位置
911     List<int> targetIndices = new List<int>(availableIndices);
912
913     // 困难模式下，尝试让卡牌离原位置更远
914     if (isHard && originalGridCoords.Count == children.Count)
915     {
916         // 为困难模式创建一个偏向于远距离交换的映射
917         System.Random distanceRng = new System.Random(seed + 3);
918
919         // 多次尝试优化位置，使卡牌移动得更远
920         for (int attempt = 0; attempt < 3; attempt++)
921         {
922             for (int i = 0; i < targetIndices.Count - 1; i++)
923             {
924                 int currentIndex = availableIndices[i];
925                 int currentTarget = targetIndices[i];
926
927                 // 检查当前位置和目标位置的距离
928                 if (currentIndex < originalGridCoords.Count && currentTarget < originalGridCoords.Count &&
929                     originalGridCoords[currentIndex].x >= 0 && originalGridCoords[currentTarget].x >= 0)
930                 {
931                     int currentDistance = CalculateManhattanDistance(
932                         originalGridCoords[currentIndex].x, originalGridCoords[currentIndex].y,
933                         originalGridCoords[currentTarget].x, originalGridCoords[currentTarget].y);
934
935                     // 寻找一个更远的目标位置
936                     for (int j = i + 1; j < targetIndices.Count; j++)
937                     {
938                         int otherIndex = availableIndices[j];
939                         int otherTarget = targetIndices[j];
940
941                         if (otherIndex < originalGridCoords.Count && otherTarget < originalGridCoords.Count &&
942                             originalGridCoords[otherIndex].x >= 0 && originalGridCoords[otherTarget].x >= 0)
943                         {
944                             int newDistance1 = CalculateManhattanDistance(
945                                 originalGridCoords[currentIndex].x, originalGridCoords[currentIndex].y,
946                                 originalGridCoords[otherTarget].x, originalGridCoords[otherTarget].y);
947
948                             int newDistance2 = CalculateManhattanDistance(
949                                 originalGridCoords[otherIndex].x, originalGridCoords[otherIndex].y,
```

```
950         originalGridCoords[currentTarget].x, originalGridCoords[currentTarget].y);
951
952         // 如果交换后两个卡牌都离原位置更远, 则进行交换
953         if (newDistance1 > currentDistance && newDistance2 >
954             CalculateManhattanDistance(
955                 originalGridCoords[otherIndex].x, originalGridCoords[otherIndex].y,
956                 originalGridCoords[otherTarget].x, originalGridCoords[otherTarget].y))
957         {
958             // 交换目标索引
959             int temp = targetIndices[i];
960             targetIndices[i] = targetIndices[j];
961             targetIndices[j] = temp;
962             break;
963         }
964     }
965 }
966 }
967 }
968 }
969 }
970 else
971 {
972     // 普通模式或无法计算网格坐标时, 使用原有随机方式
973     System.Random targetRng = new System.Random(seed + 2);
974     for (int i = targetIndices.Count - 1; i > 0; i--)
975     {
976         int j = targetRng.Next(0, i + 1);
977         int temp = targetIndices[i];
978         targetIndices[i] = targetIndices[j];
979         targetIndices[j] = temp;
980     }
981 }
982
983 // 应用位置变化
984 for (int i = 0; i < children.Count; i++)
985 {
986     if (keepIndices.Contains(i))
987     {
988         // 保持原位
989         continue;
990     }
991
992     // 找到当前索引在availableIndices中的位置
993     int indexInAvailable = availableIndices.IndexOf(i);
994     if (indexInAvailable >= 0 && indexInAvailable < targetIndices.Count)
995     {
996         int targetIndex = targetIndices[indexInAvailable];
997         children[i].anchoredPosition = originalPositions[targetIndex];
998     }
999 }
```

```
1000
1001     RefreshAllPositionIndices();
1002
1003     // 更新边框显示状态
1004     UpdateBorderVisibility();
1005 }
1006 }

// File: client/Assets/Scripts/DraggableGridItem.cs
1  using System.Collections.Generic;
2  using UnityEngine;
3  using UnityEngine.UI;
4  using UnityEngine.EventSystems;
5  using DG.Tweening; // 引入DOTween
6  using System.Collections;
7  using TMPro;
8  public class DraggableGridItem : MonoBehaviour, IBeginDragHandler, IDragHandler, IEndDragHandler
9  {
10     public RectTransform rectTransform
11     {
12         get
13         {
14             return GetComponent<RectTransform>();
15         }
16     }
17     public Canvas canvas;
18     //private Vector2 originalPosition;
19     private static int originalPositionIndex; // 记录原始位置索引
20     public static int targetPositionIndex;
21     public static Vector2 targetPosition
22     {
23         get
24         {
25             return new Vector2((targetPositionIndex / wid) * carWid, (targetPositionIndex % hei) * carHei);
26         }
27     }
28     static Vector2 originalPosition
29     {
30         get
31         {
32             return new Vector2((originalPositionIndex / wid) * carWid, (originalPositionIndex % hei) * carHei);
33         }
34     }
35     Vector2 vecPosition
36     {
37         get
38         {
39             return new Vector2((PositionIndex/wid)*carWid,(PositionIndex%hei)*carHei);
40         }
41     }
```

```
42     public static float carWid
43     {
44         get
45         {
46             return picmgr.instance.carWid;
47         }
48     }
49     public static float carHei
50     {
51         get
52         {
53             return picmgr.instance.carHei;
54         }
55     }
56     public static int wid
57     {
58         get
59         {
60             return picmgr.instance.width;
61         }
62     }
63     public static int hei
64     {
65         get
66         {
67             return picmgr.instance.height;
68         }
69     }
70     public int originalSiblingIndex { get; private set; } // 原始的层级索引，公开供其他脚本访问
71     public static bool isAnyItemDragging = false; // 静态变量，标记是否有任何卡牌正在被拖拽
72     public int PositionIndex { get; set; } // 当前卡牌的位置索引
73     // 在 Inspector 中显示的邻居合法状态（true 表示在对应方向存在邻居/相邻）
74     public bool adjacentLeft = false;
75     public bool adjacentRight = false;
76     public bool adjacentTop = false;
77     public bool adjacentBottom = false;
78
79     // 可由外部调用以一次性更新所有邻居状态
80     public void SetAdjacency(bool left, bool right, bool top, bool bottom)
81     {
82         adjacentLeft = left;
83         adjacentRight = right;
84         adjacentTop = top;
85         adjacentBottom = bottom;
86     }
87     // 拖拽时一起移动的组
88     private List<DraggableGridItem> dragGroup = null;
89     private Dictionary<DraggableGridItem, Vector2> groupOriginalPositions = null;
90     private Dictionary<DraggableGridItem, int> groupOriginalSiblingIndices = null;
91     private bool isDragging = false; // 标记是否正在拖拽
```

```
92     public bool IsBeingDragged => isDragging;
93     internal Texture2D pic;
94     internal float uvWidth;
95     internal float uvHeight;
96     internal float uvY;
97     internal float uvX;
98
99
100    public void OnBeginDrag(PointerEventData eventData)
101    {
102        // 发牌动画未完成，禁止拖拽
103        if (picmgr.instance != null && !picmgr.instance.cardsReady)
104        {
105            return;
106        }
107
108        // 如果已经有卡牌在拖拽，则直接返回，禁止同时拖拽多张卡牌
109        if (isAnyItemDragging)
110        {
111            return;
112        }
113
114        // 设置静态拖拽状态为true
115        isAnyItemDragging = true;
116
117        // 记录原始位置和层级
118        originalPositionIndex = PositionIndex;
119
120        originalSiblingIndex = transform.GetSiblingIndex();
121
122        // 收集和本格连接的邻居，一起移动
123        CollectConnectedGroup();
124
125        // 设置拖拽状态为true
126        isDragging = true;
127
128        // 将组内所有项置于最上层，记录原始层级和位置
129        if (dragGroup != null)
130        {
131            foreach (var it in dragGroup)
132            {
133                if (groupOriginalSiblingIndices != null && !groupOriginalSiblingIndices.ContainsKey(it))
134                    groupOriginalSiblingIndices[it] = it.transform.GetSiblingIndex();
135                if (groupOriginalPositions != null && !groupOriginalPositions.ContainsKey(it))
136                    groupOriginalPositions[it] = it.rectTransform.anchoredPosition;
137                it.transform.SetAsLastSibling();
138            }
139        }
140    }
141
```

```
142 public void OnDrag(PointerEventData eventData)
143 {
144     RectTransform parentRect = transform.parent as RectTransform;
145     if (parentRect == null) return;
146
147     Vector2 rawNewPos = rectTransform.anchoredPosition + eventData.delta / canvas.scaleFactor;
148     Vector2 rawDelta = rawNewPos - originalPosition;
149
150     List<DraggableGridItem> allItems = dragGroup ?? new List<DraggableGridItem>() { this };
151
152     float minDX = float.MinValue, maxDX = float.MaxValue;
153     float minDY = float.MinValue, maxDY = float.MaxValue;
154
155     foreach (var item in allItems)
156     {
157         Vector2 orig;
158         if (item == this)
159             orig = originalPosition;
160         else if (groupOriginalPositions != null && groupOriginalPositions.TryGetValue(item, out Vector2 o))
161             orig = o;
162         else
163             continue;
164
165         float iw = item.rectTransform.rect.width;
166         float ih = item.rectTransform.rect.height;
167
168         minDX = Mathf.Max(minDX, -orig.x);
169         maxDX = Mathf.Min(maxDX, parentRect.rect.width - iw - orig.x);
170         minDY = Mathf.Max(minDY, -orig.y);
171         maxDY = Mathf.Min(maxDY, parentRect.rect.height - ih - orig.y);
172     }
173
174     Vector2 clampedDelta = new Vector2(
175         Mathf.Clamp(rawDelta.x, minDX, maxDX),
176         Mathf.Clamp(rawDelta.y, minDY, maxDY)
177     );
178
179     rectTransform.anchoredPosition = originalPosition + clampedDelta;
180
181     if (dragGroup != null && groupOriginalPositions != null)
182     {
183         foreach (var it in dragGroup)
184         {
185             if (it == this) continue;
186             if (groupOriginalPositions.TryGetValue(it, out Vector2 orig))
187             {
188                 it.rectTransform.anchoredPosition = orig + clampedDelta;
189             }
190         }
191     }
```



```
192
193 // 拖拽过程中跳过 UpdateBorderVisibility, 避免每帧大量 GetComponent/FindChildByName 导致卡顿
194 // 边框视觉装饰不需要实时刷新, 在 OnEndDrag 时统一更新即可
195
196 var diff = rectTransform.anchoredPosition - vecPosition;
197 if (Mathf.Abs(diff.x) > carWid/2 ||
198     Mathf.Abs(diff.y) > carHei/2)
199 {
200     int x = Mathf.RoundToInt(rectTransform.anchoredPosition.x / carWid);
201     int y = Mathf.RoundToInt(rectTransform.anchoredPosition.y / carHei);
202
203     x = Mathf.Min(wid, Mathf.Max(0, x));
204     y = Mathf.Min(hei, Mathf.Max(0, y));
205     targetPositionIndex = x*hei + y;
206 }
207 else
208 {
209     targetPositionIndex = PositionIndex;
210
211 }
212 setText($"{PositionIndex} to {targetPositionIndex}");
213 }
214 void setText(string c)
215 {
216     var m = GetComponentInChildren<TextMeshProUGUI>();
217     if(m!=null)
218         m.text = c;
219 }
220
221 // 检查拖拽组是否在父对象边界内
222 private bool IsGroupWithinParentBounds(Dictionary<DraggableGridItem, Vector2> potentialPositions = null)
223 {
224     // 获取父对象的RectTransform
225     RectTransform parentRect = transform.parent as RectTransform;
226     if (parentRect == null) return false;
227
228     // 获取父对象的世界空间边界
229     Rect parentWorldRect = parentRect.rect;
230
231     // 确定要检查的项目集合
232     IEnumerable<DraggableGridItem> itemsToCheck;
233     if (potentialPositions != null)
234     {
235         // 如果提供了潜在位置, 使用该字典中的所有项
236         itemsToCheck = potentialPositions.Keys;
237     }
238     else if (dragGroup != null)
239     {
240         // 否则使用当前拖拽组中的项
241         itemsToCheck = dragGroup;
```

```
242     }
243     else
244     {
245         // 如果没有拖拽组，只有当前项
246         itemsToCheck = new List<DraggableGridItem> { this };
247     }
248
249     // 检查组内所有项是否都在父对象边界内
250     foreach (var item in itemsToCheck)
251     {
252         RectTransform itemRect = item.rectTransform;
253
254         // 获取项的位置
255         Vector2 itemPosition;
256         if (potentialPositions != null && potentialPositions.TryGetValue(item, out Vector2 pos))
257         {
258             // 使用潜在位置
259             itemPosition = pos;
260         }
261         else
262         {
263             // 使用当前位置
264             itemPosition = itemRect.anchoredPosition;
265         }
266
267         // 计算项的边界
268         float itemLeft = itemPosition.x - itemRect.rect.width / 2;
269         float itemRight = itemPosition.x + itemRect.rect.width / 2;
270         float itemTop = itemPosition.y + itemRect.rect.height / 2;
271         float itemBottom = itemPosition.y - itemRect.rect.height / 2;
272
273         // 检查是否超出父对象边界
274         if (itemLeft < parentWorldRect.xMin ||
275             itemRight > parentWorldRect.xMax ||
276             itemBottom < parentWorldRect.yMin ||
277             itemTop > parentWorldRect.yMax)
278         {
279             return false;
280         }
281     }
282     return true;
283 }
284
285 public void Turn(bool atonce=false)
286 {
287     if (atonce)
288     {
289         settex();
290     }
291     else
```

```
292     {
293
294         transform.DOScaleX(0, 0.25f).onComplete = () =>
295         {
296             transform.localScale = new Vector3(0, 1, 1);
297             settex();
298             transform.DOScaleX(1, 0.25f).onComplete = () =>
299             {
300                 transform.localScale = new Vector3(1, 1, 1);
301             };
302         };
303
304
305         var x = transform.localPosition.x;
306         float wid = transform.GetComponent<RectTransform>().rect.width;
307         transform.DOLocalMoveX(x + wid / 2, 0.25f).onComplete = () =>
308         {
309             var p = transform.localPosition;
310             p.x = x + wid / 2;
311             transform.localPosition = p;
312
313             transform.DOLocalMoveX(x, 0.25f).onComplete = () =>
314             {
315                 var p = transform.localPosition;
316                 p.x = x;
317                 transform.localPosition = p;
318             };
319         };
320
321     }
322
323 }
324
325 internal void settex()
326 {
327     rawImage.color = Color.white;
328     rawImage.uvRect = new Rect(uvX, uvY, uvWidth, uvHeight);
329     rawImage.texture = pic;
330 }
331
332 RawImage rawImage
333 {
334     get
335     {
336         return GetComponent<RawImage>();
337     }
338 }
339 public void OnEndDrag(PointerEventData eventData)
340 {
341
```

```
342     bool shouldSwap = false;
343
344     if(originalPositionIndex != targetPositionIndex)
345     {
346         // 条件2: 所有拖拽组成员位置是否合法
347         bool isPositionValid = CheckPotentialPositionValidity();
348         if (isPositionValid)
349         {
350             shouldSwap = true;
351         }
352     }
353     if (shouldSwap)
354     {
355         // 执行交换位置
356         SwapPositions(targetPosition);
357     }
358     else
359     {
360         // 拖拽失败, 整个组退回原位置
361         ResetAllDraggedItemsToOriginalPosition();
362     }
363
364     dragGroup = null;
365     groupOriginalPositions = null;
366     groupOriginalSiblingIndices = null;
367
368     isDragging = false;
369
370     // 不要立即重置静态拖拽状态, 等待动画完成后再重置
371 }
372
373 // 将所有拖拽的项重置到原始位置
374 private void ResetAllDraggedItemsToOriginalPosition()
375 {
376     // 对组内每个元素做回退动画
377     if (dragGroup != null && groupOriginalPositions != null)
378     {
379         Sequence seq = DOTween.Sequence();
380         foreach (var it in dragGroup)
381         {
382             if (groupOriginalPositions.TryGetValue(it, out Vector2 orig))
383             {
384                 seq.Join(it.rectTransform.DOAnchorPos(orig, 0.25f));
385                 // 重置位置索引
386                 int index = GetIndexFromAnchoredPosition(orig);
387                 it.PositionIndex = index;
388             }
389         }
390         seq.OnComplete(() => {
391             // 恢复原始层级
```

```
392         if (groupOriginalSiblingIndices != null)
393         {
394             foreach (var kv in groupOriginalSiblingIndices)
395             {
396                 kv.Key.transform.SetSiblingIndex(kv.Value);
397             }
398         }
399         // 更新边框显示状态
400         picmgr picManager = GetComponentInParent<picmgr>();
401         if (picManager != null)
402         {
403             picManager.RefreshAllPositionIndices();
404             picManager.UpdateBorderVisibility();
405         }
406         // 动画完成，重置静态拖拽状态，允许新的操作
407         isAnyItemDragging = false;
408     });
409 }
410 else
411 {
412     // 单个元素回退
413     Tween tween = rectTransform.DOAnchorPos(originalPosition, 0.3f);
414     tween.OnComplete(() => {
415         rectTransform.anchoredPosition = originalPosition;
416         transform.SetSiblingIndex(originalSiblingIndex);
417         // 重置位置索引
418         PositionIndex = originalPositionIndex;
419         picmgr picManager = GetComponentInParent<picmgr>();
420         if (picManager != null)
421         {
422             picManager.RefreshAllPositionIndices();
423             picManager.UpdateBorderVisibility();
424         }
425         // 动画完成，重置静态拖拽状态，允许新的操作
426         isAnyItemDragging = false;
427     });
428 }
429 }
430
431 void Reset()
432 {
433
434 }
435
436
437 // 检查拖拽组挪到潜在目标位置后是否合法
438 private bool CheckPotentialPositionValidity(Vector2 potentialTargetPos = default)
439 {
440     // 准备拖拽组（源组）
441     List<DraggableGridItem> dragGroupList = (dragGroup != null && dragGroup.Count > 0) ? new List<DraggableGridItem>(dragGrou ...
```

```
442
443     RectTransform parentRect = transform.parent as RectTransform;
444     if (parentRect == null) return false;
445
446     // 获取父对象的边界
447     Rect parentWorldRect = new Rect(0, 0, parentRect.rect.width, parentRect.rect.height);
448
449     // 计算拖拽组移动后的目标位置集合，并检查每个位置是否合法
450     foreach (var item in dragGroupList)
451     {
452         Vector2 newPosition = item.rectTransform.anchoredPosition+new Vector2(item.rectTransform.sizeDelta.x/2,item.rectTransfor ...
453         if(!parentWorldRect.Contains(newPosition))
454         {
455             return false;
456         }
457     }
458
459     return true;
460 }
461
462 // 交换位置
463 private void SwapPositions(Vector2 targetPos)
464 {
465     // 更新当前卡牌的位置索引
466     int newIndex = GetIndexFromAnchoredPosition(targetPos);
467     PositionIndex = newIndex;
468     // 准备拖拽组（源组）
469     List<DraggableGridItem> dragGroupList = (dragGroup != null && dragGroup.Count > 0) ? new List<DraggableGridItem>{(dragGrou ...
470
471     // 确保 source 原始位置/索引可用
472     Dictionary<DraggableGridItem, Vector2> originalPositions = new Dictionary<DraggableGridItem, Vector2>();
473     Dictionary<DraggableGridItem, int> originalSiblings = new Dictionary<DraggableGridItem, int>();
474
475     // 记录拖拽组所有项的原始位置和层级
476     foreach (var item in dragGroupList)
477     {
478         if (groupOriginalPositions != null && groupOriginalPositions.TryGetValue(item, out Vector2 p))
479             originalPositions[item] = p;
480         else
481             originalPositions[item] = item.rectTransform.anchoredPosition;
482         originalSiblings[item] = item.transform.GetSiblingIndex();
483     }
484
485     // 获取拖拽源项（当前拖拽的卡牌）的原始位置
486     Vector2 dragSourcePosition = originalPositions[this];
487
488     // 计算拖拽组移动后的目标位置集合
489     Dictionary<DraggableGridItem, Vector2> targetPositions = new Dictionary<DraggableGridItem, Vector2>();
490     foreach (var item in dragGroupList)
491     {
```

```
133     /// <summary>
134     /// 安全地获取bool值, 如果转换失败则返回默认值
135     /// </summary>
136     /// <param name="data">JsonData对象</param>
137     /// <param name="key">键名</param>
138     /// <param name="defaultValue">默认值</param>
139     /// <returns>bool值或默认值</returns>
140     public static bool GetBool(this JsonData data, string key, bool defaultValue = false)
141     {
142         if (data == null || !data.IsObject) return defaultValue;
143
144         try
145         {
146             // 检查键是否存在
147             if (data.ContainsKey(key))
148             {
149                 JsonData value = data[key];
150                 if (value.IsBoolean)
151                     return (bool)value;
152
153                 if (value.IsInt)
154                     return (int)value != 0;
155
156                 if (value.IsLong)
157                     return (long)value != 0L;
158
159                 if (value.IsDouble)
160                     return Math.Abs((double)value) > double.Epsilon;
161
162                 if (value.IsString)
163                 {
164                     string str = ((string)value).ToLower();
165                     if (str == "true" || str == "1")
166                         return true;
167                     if (str == "false" || str == "0")
168                         return false;
169                 }
170             }
171         }
172         catch
173         {
174             // 转换失败, 返回默认值
175         }
176
177         return defaultValue;
178     }
179
180     /// <summary>
181     /// 安全地获取float值, 如果转换失败则返回默认值
182     /// </summary>
```

```
183     /// <param name="data">JsonData对象</param>
184     /// <param name="key">键名</param>
185     /// <param name="defaultValue">默认值</param>
186     /// <returns>float值或默认值</returns>
187     public static float GetFloat(this JsonData data, string key, float defaultValue = 0f)
188     {
189         if (data == null || !data.IsObject) return defaultValue;
190
191         try
192         {
193             // 检查键是否存在
194             if (data.ContainsKey(key))
195             {
196                 JsonData value = data[key];
197                 if (value.IsDouble)
198                     return (float)(double)value;
199
200                 if (value.IsInt)
201                     return (float)(int)value;
202
203                 if (value.IsLong)
204                     return (float)(long)value;
205
206                 if (value.IsString)
207                 {
208                     float result;
209                     if (float.TryParse((string)value, out result))
210                         return result;
211                 }
212
213                 if (value.IsBoolean)
214                     return (bool)value ? 1f : 0f;
215             }
216         }
217         catch
218         {
219             // 转换失败, 返回默认值
220         }
221
222         return defaultValue;
223     }
224
225     /// <summary>
226     /// 安全地获取double值, 如果转换失败则返回默认值
227     /// </summary>
228     /// <param name="data">JsonData对象</param>
229     /// <param name="key">键名</param>
230     /// <param name="defaultValue">默认值</param>
231     /// <returns>double值或默认值</returns>
232     public static double GetDouble(this JsonData data, string key, double defaultValue = 0.0)
```



```
233     {
234         if (data == null || !data.IsObject) return defaultValue;
235
236         try
237         {
238             // 检查键是否存在
239             if (data.ContainsKey(key))
240             {
241                 JsonData value = data[key];
242                 if (value.IsDouble)
243                     return (double)value;
244
245                 if (value.IsInt)
246                     return (double)(int)value;
247
248                 if (value.IsLong)
249                     return (double)(long)value;
250
251                 if (value.IsString)
252                 {
253                     double result;
254                     if (double.TryParse((string)value, out result))
255                         return result;
256                 }
257
258                 if (value.IsBoolean)
259                     return (bool)value ? 1.0 : 0.0;
260             }
261         }
262         catch
263         {
264             // 转换失败，返回默认值
265         }
266
267         return defaultValue;
268     }
269
270     /// <summary>
271     /// 安全地获取long值，如果转换失败则返回默认值
272     /// </summary>
273     /// <param name="data">JsonData对象</param>
274     /// <param name="key">键名</param>
275     /// <param name="defaultValue">默认值</param>
276     /// <returns>long值或默认值</returns>
277     public static long GetLong(this JsonData data, string key, long defaultValue = 0L)
278     {
279         if (data == null || !data.IsObject) return defaultValue;
280
281         try
282         {
```

```
283         // 检查键是否存在
284         if (data.ContainsKey(key))
285         {
286             JsonData value = data[key];
287             if (value.IsLong)
288                 return (long)value;
289
290             if (value.IsInt)
291                 return (long)(int)value;
292
293             if (value.IsDouble)
294                 return (long)(double)value;
295
296             if (value.IsString)
297             {
298                 long result;
299                 if (long.TryParse((string)value, out result))
300                     return result;
301             }
302
303             if (value.IsBoolean)
304                 return (bool)value ? 1L : 0L;
305         }
306     }
307     catch
308     {
309         // 转换失败, 返回默认值
310     }
311
312     return defaultValue;
313 }
314 }
315 }
```



```
// File: client/Assets/Scripts/Extensions/ObjectPoolManager.cs
1 using System.Collections.Generic;
2 using UnityEngine;
3
4 /// <summary>
5 /// 通用对象池管理器
6 /// </summary>
7 /// <typeparam name="T">要池化的对象类型</typeparam>
8 public class ObjectPoolManager<T> where T : Component
9 {
10     private readonly T prefab;
11     private readonly Transform poolParent;
12     private readonly Queue<T> pooledObjects;
13     private readonly bool autoExpand;
14     private readonly int maxSize;
15     private int currentCount;
```

```
16
17    /// <summary>
18    /// 构造函数
19    /// </summary>
20    /// <param name="prefab">要池化的预制体</param>
21    /// <param name="initialSize">初始池大小</param>
22    /// <param name="poolParent">池中对象的父物体</param>
23    /// <param name="autoExpand">是否自动扩容</param>
24    /// <param name="maxSize">最大池大小（-1表示无限制）</param>
25    public ObjectPoolManager(T prefab, int initialSize, Transform poolParent = null, bool autoExpand = true, int maxSize = -1)
26    {
27        this.prefab = prefab;
28        this.poolParent = poolParent;
29        this.autoExpand = autoExpand;
30        this.maxSize = maxSize;
31        this.pooledObjects = new Queue<T>();
32        this.currentCount = 0;
33
34        // 创建初始对象
35        for (int i = 0; i < initialSize; i++)
36        {
37            CreateObject();
38        }
39    }
40
41    /// <summary>
42    /// 从对象池中获取一个对象
43    /// </summary>
44    /// <returns>对象实例</returns>
45    public T GetObject()
46    {
47        T obj;
48        if (pooledObjects.Count > 0)
49        {
50            obj = pooledObjects.Dequeue();
51        }
52        else if (autoExpand && (maxSize == -1 || currentCount < maxSize))
53        {
54            obj = CreateObject();
55        }
56        else
57        {
58            return null;
59        }
60
61        obj.gameObject.SetActive(true);
62        return obj;
63    }
64
65    /// <summary>
```

```
66    /// 将对象返回到对象池
67    /// </summary>
68    /// <param name="obj">要回收的对象</param>
69    public void ReturnObject(T obj)
70    {
71        if (obj == null) return;
72
73        // 如果池已满且有大小限制, 则销毁对象
74        if (maxSize != -1 && currentCount > maxSize)
75        {
76            Object.Destroy(obj.gameObject);
77            currentCount--;
78            return;
79        }
80
81        obj.gameObject.SetActive(false);
82        obj.transform.SetParent(poolParent);
83        pooledObjects.Enqueue(obj);
84    }
85
86    /// <summary>
87    /// 创建新对象并加入池中
88    /// </summary>
89    /// <returns>新创建的对象</returns>
90    private T CreateObject()
91    {
92        T obj = Object.Instantiate(prefab);
93        obj.gameObject.SetActive(false);
94        if (poolParent != null)
95        {
96            obj.transform.SetParent(poolParent);
97        }
98        pooledObjects.Enqueue(obj);
99        currentCount++;
100        return obj;
101    }
102
103    /// <summary>
104    /// 预加载指定数量的对象
105    /// </summary>
106    /// <param name="count">预加载数量</param>
107    public void Preload(int count)
108    {
109        int spaceAvailable = maxSize == -1 ? count : Mathf.Min(count, maxSize - currentCount);
110        for (int i = 0; i < spaceAvailable; i++)
111        {
112            CreateObject();
113        }
114    }
115
```

```
116    /// <summary>
117    /// 获取当前池中可用对象数量
118    /// </summary>
119    public int AvailableCount => pooledObjects.Count;
120
121    /// <summary>
122    /// 获取当前池中总对象数量
123    /// </summary>
124    public int TotalCount => currentCount;
125 }
126
127 /// <summary>
128 /// GameObject专用对象池管理器
129 /// </summary>
130 public class GameObjectPoolManager
131 {
132     private readonly GameObject prefab;
133     private readonly Transform poolParent;
134     private readonly Queue<GameObject> pooledObjects;
135     private readonly bool autoExpand;
136     //private readonly int maxSize;
137     private int currentCount;
138
139     /// <summary>
140     /// 构造函数
141     /// </summary>
142     /// <param name="prefab">要池化的预制体</param>
143     /// <param name="initialSize">初始池大小</param>
144     /// <param name="poolParent">池中对象的父物体</param>
145     /// <param name="autoExpand">是否自动扩容</param>
146     /// <param name="maxSize">最大池大小（-1表示无限制）</param>
147     public GameObjectPoolManager(GameObject prefab, int initialSize, Transform poolParent = null, bool autoExpand = true, int max ...
148     {
149         this.prefab = prefab;
150         this.poolParent = poolParent;
151         this.autoExpand = autoExpand;
152         //this.maxSize = maxSize;
153         this.pooledObjects = new Queue<GameObject>();
154         this.currentCount = 0;
155
156         // 创建初始对象
157         for (int i = 0; i < initialSize; i++)
158         {
159             CreateObject();
160         }
161     }
162
163     /// <summary>
164     /// 从对象池中获取一个对象
165     /// </summary>
```

```
166    /// <returns>对象实例</returns>
167    public GameObject GetObject()
168    {
169        GameObject obj;
170        if (pooledObjects.Count > 0)
171        {
172            obj = pooledObjects.Dequeue();
173        }
174        else
175        {
176            obj = CreateObject();
177        }
178
179        obj.SetActive(true);
180        return obj;
181    }
182
183    /// <summary>
184    /// 将对象返回到对象池
185    /// </summary>
186    /// <param name="obj">要回收的对象</param>
187    public void ReturnObject(GameObject obj)
188    {
189        if (obj == null) return;
190
191        // 如果池已满且有大小限制，则销毁对象
192        //if (maxSize != -1 && currentCount > maxSize)
193        //{
194            // Object.Destroy(obj);
195            // currentCount--;
196            // return;
197        //}
198
199        obj.SetActive(false);
200        obj.transform.SetParent(poolParent);
201        pooledObjects.Enqueue(obj);
202    }
203
204    /// <summary>
205    /// 创建新对象并加入池中
206    /// </summary>
207    /// <returns>新创建的对象</returns>
208    private GameObject CreateObject()
209    {
210        GameObject obj = Object.Instantiate(prefab);
211        obj.SetActive(false);
212        if (poolParent != null)
213        {
214            obj.transform.SetParent(poolParent);
215        }
216    }
```

```
216     pooledObjects.Enqueue(obj);
217     currentCount++;
218     return obj;
219 }
220
221 /// <summary>
222 /// 预加载指定数量的对象
223 /// </summary>
224 /// <param name="count">预加载数量</param>
225 public void Preload(int count)
226 {
227     int spaceAvailable = count ;
228     for (int i = 0; i < spaceAvailable; i++)
229     {
230         CreateObject();
231     }
232 }
233
234 /// <summary>
235 /// 获取当前池中可用对象数量
236 /// </summary>
237 public int AvailableCount => pooledObjects.Count;
238
239 /// <summary>
240 /// 获取当前池中总对象数量
241 /// </summary>
242 public int TotalCount => currentCount;
243 }

// File: client/Assets/Scripts/Extensions/TipsPoolManager.cs
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4  using TMPro;
5
6  /// <summary>
7  /// Tips对象池管理器
8  /// </summary>
9  public class TipsPoolManager : MonoBehaviour
10 {
11     [Header("普通提示预制体")]
12     public GameObject normalTipsPrefab;
13
14     [Header("城墙提示预制体")]
15     public GameObject wallTipsPrefab;
16
17     [Header("对象池设置")]
18     public int initialPoolSize = 10;
19     public int maxPoolSize = 50;
20
```

```
21 private GameObjectPoolManager normalTipsPool;
22 private GameObjectPoolManager bigTipsPool;
23 private Transform poolParent;
24
25 private static TipsPoolManager _instance;
26 public static TipsPoolManager Instance
27 {
28     get
29     {
30         if (_instance == null)
31         {
32             _instance = FindObjectOfType<TipsPoolManager>();
33             if (_instance == null)
34             {
35                 GameObject obj = new GameObject("TipsPoolManager");
36                 _instance = obj.AddComponent<TipsPoolManager>();
37             }
38         }
39         return _instance;
40     }
41 }
42
43 private void Awake()
44 {
45     if (_instance == null)
46     {
47         _instance = this;
48         DontDestroyOnLoad(gameObject);
49         // 初始化对象池将在InitializePools方法中进行
50     }
51     else if (_instance != this)
52     {
53         Destroy(gameObject);
54     }
55 }
56
57 /// <summary>
58 /// 初始化对象池
59 /// </summary>
60 public void InitializePools()
61 {
62     // 如果已经初始化过，则先清理
63     if (poolParent != null)
64     {
65         Destroy(poolParent.gameObject);
66     }
67
68     // 创建池的父物体
69     poolParent = new GameObject("TipsPool").transform;
70     poolParent.SetParent(transform);
```



```
71
72     // 初始化普通提示对象池
73     if (normalTipsPrefab != null)
74     {
75         normalTipsPool = new GameObjectPoolManager(normalTipsPrefab, initialPoolSize, poolParent, true, maxPoolSize);
76     }
77
78     // 初始化城墙提示对象池
79     if (wallTipsPrefab != null)
80     {
81         bigTipsPool = new GameObjectPoolManager(wallTipsPrefab, initialPoolSize, poolParent, true, maxPoolSize);
82     }
83 }
84
85 /// <summary>
86 /// 获取普通提示对象
87 /// </summary>
88 /// <returns>普通提示对象</returns>
89 public GameObject GetNormalTips()
90 {
91     if (normalTipsPool != null)
92     {
93         return normalTipsPool.GetObject();
94     }
95     return null;
96 }
97
98 /// <summary>
99 /// 获取大提示，红色的
100 /// </summary>
101 /// <returns>红色的对象</returns>
102 public GameObject GetBigTips()
103 {
104     if (bigTipsPool != null)
105     {
106         return bigTipsPool.GetObject();
107     }
108     return null;
109 }
110
111 /// <summary>
112 /// 回收普通提示对象
113 /// </summary>
114 /// <param name="tipsObject">要回收的普通提示对象</param>
115 public void ReturnNormalTips(GameObject tipsObject)
116 {
117     if (normalTipsPool != null && tipsObject != null)
118     {
119         // 重置TextMeshPro组件的内容和属性
120         var textMeshPro = tipsObject.GetComponent<TextMeshProUGUI>();
```

```
121         if (textMeshPro != null)
122         {
123             textMeshPro.text = "";
124             textMeshPro.fontSize = 36; // 重置为默认小字体大小
125         }
126         tipsObject.transform.localScale = Vector3.one;
127         normalTipsPool.ReturnObject(tipsObject);
128     }
129 }
130
131 /// <summary>
132 /// 回收城墙提示对象
133 /// </summary>
134 /// <param name="tipsObject">要回收的城墙提示对象</param>
135 public void ReturnBigTips(GameObject tipsObject)
136 {
137     if (bigTipsPool != null && tipsObject != null)
138     {
139         // 重置TextMeshPro组件的内容和属性
140         var textMeshPro = tipsObject.GetComponent<TextMeshProUGUI>();
141         if (textMeshPro != null)
142         {
143             textMeshPro.text = "";
144             textMeshPro.fontSize = 36; // 重置为默认小字体大小
145         }
146         tipsObject.transform.localScale = Vector3.one;
147         bigTipsPool.ReturnObject(tipsObject);
148     }
149 }
150
151 /// <summary>
152 /// 预加载指定数量的对象
153 /// </summary>
154 /// <param name="normalCount">普通提示预加载数量</param>
155 /// <param name="wallCount">城墙提示预加载数量</param>
156 public void PreloadTips(int normalCount, int wallCount)
157 {
158     if (normalTipsPool != null)
159     {
160         normalTipsPool.Preload(normalCount);
161     }
162
163     if (bigTipsPool != null)
164     {
165         bigTipsPool.Preload(wallCount);
166     }
167 }
168 }
```

// File: client/Assets/Scripts/Extensions/TipsPoolInitializer.cs

```
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4  using TMPro;
5  using UnityEngine.UI;
6
7  /// <summary>
8  /// Tips对象池初始化器
9  /// 用于在运行时创建tips预制件并初始化对象池
10 /// </summary>
11 public class TipsPoolInitializer : MonoBehaviour
12 {
13     [Header("Tips UI设置")]
14     public Font font;
15     public Color normalTipsColor = Color.white;
16     public Color wallTipsColor = Color.red;
17     public int fontSize = 36;
18
19     [Header("对象池设置")]
20     public int initialNormalTipsCount = 10;
21     public int initialWallTipsCount = 5;
22
23     private TipsPoolManager tipsPoolManager;
24
25     void Awake()
26     {
27         InitializeTipsPool();
28     }
29
30     /// <summary>
31     /// 初始化Tips对象池
32     /// </summary>
33     private void InitializeTipsPool()
34     {
35         // 获取或创建TipsPoolManager实例
36         tipsPoolManager = TipsPoolManager.Instance;
37
38         // 创建普通提示预制件
39         if (tipsPoolManager.normalTipsPrefab == null)
40         {
41             GameObject normalTipsPrefab = CreateTipsPrefab("NormalTipsPrefab", normalTipsColor);
42             tipsPoolManager.normalTipsPrefab = normalTipsPrefab;
43         }
44
45         // 创建城墙提示预制件
46         if (tipsPoolManager.wallTipsPrefab == null)
47         {
48             GameObject wallTipsPrefab = CreateTipsPrefab("WallTipsPrefab", wallTipsColor);
49             tipsPoolManager.wallTipsPrefab = wallTipsPrefab;
50         }
51     }
52 }
```

```
51
52     // 初始化对象池
53     tipsPoolManager.InitializePools();
54
55     // 预加载对象
56     tipsPoolManager.PreloadTips(initialNormalTipsCount, initialWallTipsCount);
57 }
58
59 /// <summary>
60 /// 创建Tips预制件
61 /// </summary>
62 /// <param name="name">预制件名称</param>
63 /// <param name="color">文本颜色</param>
64 /// <returns>创建的预制件对象</returns>
65 private GameObject CreateTipsPrefab(string name, Color color)
66 {
67     // 创建空的游戏对象
68     GameObject tipsPrefab = new GameObject(name);
69
70     // 添加RectTransform组件
71     RectTransform rectTransform = tipsPrefab.AddComponent<RectTransform>();
72     rectTransform.sizeDelta = new Vector2(100, 50);
73
74     // 添加CanvasRenderer组件
75     tipsPrefab.AddComponent<CanvasRenderer>();
76
77     // 添加TextMeshProUGUI组件
78     TextMeshProUGUI textMeshPro = tipsPrefab.AddComponent<TextMeshProUGUI>();
79     textMeshPro.text = "";
80     textMeshPro.fontSize = fontSize;
81     textMeshPro.color = color;
82     textMeshPro.alignment = TextAlignmentOptions.Center;
83
84     // 如果指定了字体，则设置字体
85     if (font != null)
86     {
87         // 注意：TMP字体设置需要使用TMP_FontAsset，而不是普通Font
88         // 这里仅作示意，实际使用时需要TMP字体资源
89     }
90
91     return tipsPrefab;
92 }
93 }

// File: client/Assets/Scripts/ui/frm_chapter.cs
1 using System;
2 using System.Collections;
3 using System.Collections.Generic;
4 using TMPro;
5 using UnityEngine;
```

```
6  using UnityEngine.UI;
7
8  #if UNITY_WEBGL && MINIGAME_SUBPLATFORM_DOUYIN
9  using TTSDK;
10 #endif
11
12 public class frm_chapter : frmbase
13 {
14     private const float CardAspect = 102f / 148f;
15     private const float SpacingXRatio = 0.18f;
16     private const float SpacingYRatio = 0.12f;
17     private const float MinSpacingX = 18f;
18     private const float MinSpacingY = 16f;
19     private const float MaxCardWidth = 118f;
20
21     public RectTransform chaptercontent;
22     public Button btn;
23     public Button btnsetup;
24     public Button btn_addpower;
25     public TextMeshProUGUI staminaText, levelname;
26     public power mpower;
27     public string rewardedVideoAdId = "3snlsbdejtn8h73ih6";
28
29     private TextMeshProUGUI btnText;
30     private bool isShowingRewardedVideo;
31     private int rewardedVideoRequestId;
32     private Coroutine rewardedVideoTimeoutCoroutine;
33     private const float RewardedVideoTimeoutSeconds = 30f;
34
35 #if UNITY_WEBGL && MINIGAME_SUBPLATFORM_DOUYIN
36     private TTRewardedVideoAd rewardedVideoAd;
37 #endif
38
39     private void Awake()
40     {
41         FindBtnText();
42
43         Main.RegistEvent("gamebegin", (x) =>
44         {
45             Main.inst.StartCoroutine(brushChapterContent());
46             show();
47             ApplyDebugPowerZero();
48             UpdateStaminaDisplay();
49             return 1;
50         });
51
52         Main.RegistEvent("onpowerChange", (x) =>
53         {
54             UpdateStaminaDisplay();
55             return null;
```

```
56     });
57     Main.RegistEvent("show_rewarded_power", (x) =>
58     {
59         ShowRewardedVideoForPower();
60         return null;
61     });
62     Main.RegistEvent("level_next", (x) =>
63     {
64         PlayerData.gd.levelid = datamgr.Instance.GetLevel(PlayerData.gd.levelid).NextLevel;
65         Main.inst.StartCoroutine(brushChapterContent());
66         show();
67         return 1;
68     });
69     Main.RegistEvent("onChapterChange", (x) =>
70     {
71         Main.inst.StartCoroutine(brushChapterContent());
72         show();
73         return 1;
74     });
75     Main.RegistEvent("level_back", (x) =>
76     {
77         show();
78         return null;
79     });
80     btn.onClick.AddListener(() =>
81     {
82         if (!isTurning())
83         {
84             int ret = (int)Main.DispEvent("level_play", PlayerData.gd.levelid);
85             if (ret == 1)
86             {
87                 hide();
88             }
89         }
90     });
91     btn_addpower.onClick.AddListener(() =>
92     {
93         ShowRewardedVideoForPower();
94     });
95     btnsetup.onClick.AddListener(() =>
96     {
97         Main.DispEvent("show_setup", null);
98     });
99 }
100
101 private void OnDestroy()
102 {
103     ResetRewardedVideoState(true);
104 }
105
```

```
106 private void ShowRewardedVideoForPower()
107 {
108     if (isShowingRewardedVideo)
109     {
110         Debug.LogWarning("rewardvideo busy on new request, cleanup stale ad");
111         ResetRewardedVideoState(true);
112     }
113
114     isShowingRewardedVideo = true;
115     int requestId = ++rewardedVideoRequestId;
116     StartRewardedVideoTimeout(requestId);
117
118     #if UNITY_WEBGL && MINIGAME_SUBPLATFORM_DOUYIN
119     if (Application.isEditor)
120     {
121         AddRewardedPower();
122         ResetRewardedVideoState(false);
123         return;
124     }
125
126     if (string.IsNullOrEmpty(rewardedVideoAdId))
127     {
128         ResetRewardedVideoState(false);
129         Main.DispEvent("event_msg", "激励视频广告位未配置");
130         return;
131     }
132     Debug.LogWarning("begin create rewardvideo");
133     ResetRewardedVideoAdInstance();
134
135     rewardedVideoAd = TT.CreateRewardedVideoAd(new CreateRewardedVideoAdParam
136     {
137         AdUnitId = rewardedVideoAdId,
138         Multiton = false,
139         MultitonRewardMsg = null,
140         MultitonRewardTimes = 0,
141         ProgressTip = false
142     });
143     if (rewardedVideoAd == null)
144     {
145         Debug.LogWarning("rewardvideo create returned null");
146         ResetRewardedVideoState(true);
147         Main.DispEvent("event_msg", "视频暂不可用, 请稍后再试");
148         return;
149     }
150     TTRewardedVideoAd currentAd = rewardedVideoAd;
151
152     rewardedVideoAd.OnLoad += () =>
153     {
154         if (rewardedVideoAd != currentAd || !isShowingRewardedVideo)
155         {
```

```
156         Debug.LogWarning("rewardvideo stale load ignored");
157         return;
158     }
159
160     try
161     {
162         Debug.LogWarning("rewardvideo loaded, show");
163         currentAd.Show();
164     }
165     catch (Exception e)
166     {
167         Debug.LogWarning($"Rewarded video show exception: {e.Message}");
168         ResetRewardedVideoState(true);
169         Main.DispEvent("event_msg", "视频暂不可用，请稍后再试");
170     }
171 };
172 rewardedVideoAd.OnClose += (isEnded, rewardCount) =>
173 {
174     if (rewardedVideoAd != currentAd)
175     {
176         Debug.LogWarning("rewardvideo stale close ignored");
177         return;
178     }
179
180     Debug.LogWarning($"rewardvideo close isEnded={isEnded} rewardCount={rewardCount}");
181     ResetRewardedVideoState(true);
182     if (isEnded)
183     {
184         AddRewardedPower();
185     }
186     else
187     {
188         Main.DispEvent("event_msg", "看完视频可获得体力");
189     }
190 };
191 rewardedVideoAd.OnError += (errorCode, errorMsg) =>
192 {
193     if (rewardedVideoAd != currentAd)
194     {
195         Debug.LogWarning("rewardvideo stale error ignored");
196         return;
197     }
198
199     Debug.LogWarning($"Rewarded video failed: {errorCode} {errorMsg}");
200     ResetRewardedVideoState(true);
201     Main.DispEvent("event_msg", "视频暂不可用，请稍后再试");
202 };
203 try
204 {
205     rewardedVideoAd.Load();
```



```
206     }
207     catch (Exception e)
208     {
209         Debug.LogWarning($"Rewarded video load exception: {e.Message}");
210         ResetRewardedVideoState(true);
211         Main.DispEvent("event_msg", "视频暂不可用，请稍后再试");
212     }
213 #else
214     AddRewardedPower();
215     ResetRewardedVideoState(false);
216 #endif
217 }
218
219 private void ApplyDebugPowerZero()
220 {
221     if (PlayerData.DebugForcePowerZero)
222     {
223         PlayerData.gd.power = 0;
224     }
225 }
226
227 private void StartRewardedVideoTimeout(int requestId)
228 {
229     StopRewardedVideoTimeout();
230     rewardedVideoTimeoutCoroutine = StartCoroutine(RewardedVideoTimeoutGuard(requestId));
231 }
232
233 private void StopRewardedVideoTimeout()
234 {
235     if (rewardedVideoTimeoutCoroutine == null) return;
236     StopCoroutine(rewardedVideoTimeoutCoroutine);
237     rewardedVideoTimeoutCoroutine = null;
238 }
239
240 private IEnumerator RewardedVideoTimeoutGuard(int requestId)
241 {
242     yield return new WaitForSecondsRealtime(RewardedVideoTimeoutSeconds);
243     if (!isShowingRewardedVideo || requestId != rewardedVideoRequestId) yield break;
244
245     Debug.LogWarning("rewardvideo timeout, cleanup");
246     rewardedVideoTimeoutCoroutine = null;
247     ResetRewardedVideoState(true);
248     Main.DispEvent("event_msg", "视频暂不可用，请稍后再试");
249 }
250
251 private void ResetRewardedVideoState(bool destroyAd)
252 {
253     ++rewardedVideoRequestId;
254     StopRewardedVideoTimeout();
255     isShowingRewardedVideo = false;
```

```
256
257 #if UNITY_WEBGL && MINIGAME_SUBPLATFORM_DOUYIN
258     if (destroyAd)
259     {
260         ResetRewardedVideoAdInstance();
261     }
262 #endif
263 }
264
265 #if UNITY_WEBGL && MINIGAME_SUBPLATFORM_DOUYIN
266     private void ResetRewardedVideoAdInstance()
267     {
268         if (rewardedVideoAd == null) return;
269         rewardedVideoAd.Destroy();
270         rewardedVideoAd = null;
271     }
272 #endif
273
274     private void AddRewardedPower()
275     {
276         PlayerData.gd.AddPower(20);
277         Main.DispEvent("event_msg", "体力+20");
278     }
279
280     private void FindBtnText()
281     {
282         if (btnText != null || btn == null) return;
283         btnText = btn.GetComponentInChildren<TextMeshProUGUI>();
284     }
285
286     private bool isTurning()
287     {
288         for (int i = 0; i < chaptercontent.childCount; i++)
289         {
290             if (chaptercontent.GetChild(i).GetComponent<card>().isTurning)
291                 return true;
292         }
293         return false;
294     }
295
296     protected override void OnShow()
297     {
298         base.OnShow();
299         ApplyDebugPowerZero();
300         UpdateStaminaDisplay();
301     }
302
303     private IEnumerator brushChapterContent()
304     {
305         btn.gameObject.SetActive(false);
```

```
306     levelname.gameObject.SetActive(false);
307     Main.DispEvent("event_loading", true);
308
309     var chapter = datamgr.Instance.GetChapter(PlayerData.gd.currChapter);
310
311     Texture pic = null;
312     yield return Main.inst.StartCoroutine(Main.LoadTextureFromCDN(chapter.ChapterFigure, (texture) =>
313     {
314         pic = texture;
315     }));
316
317     Main.DispEvent("event_loading", false);
318     int chapterNum = PlayerData.gd.currChapter;
319     int levelNum = PlayerData.gd.levelid % 100000;
320     levelname.text = $"{chapterNum}-{levelNum}";
321     FindBtnText();
322     if (btnText != null) btnText.text = $"挑战 {chapterNum}-{levelNum}";
323
324     for (int i = chaptercontent.childCount - 1; i >= 0; i--)
325     {
326         Transform child = chaptercontent.GetChild(i);
327         #if UNITY_EDITOR
328             if (Application.isPlaying) Destroy(child.gameObject);
329             else DestroyImmediate(child.gameObject);
330         #else
331             Destroy(child.gameObject);
332         #endif
333     }
334
335     GridLayoutGroup grid = chaptercontent.GetComponent<GridLayoutGroup>();
336     ApplyChapterGridLayout(grid, chapter.ChapterFigureX, chapter.ChapterFigureY);
337
338     for (int j = 0; j < chapter.ChapterFigureY; j++)
339     {
340         for (int i = 0; i < chapter.ChapterFigureX; i++)
341         {
342             int idx = j * chapter.ChapterFigureX + i;
343             int lid = chapter.LevelId[idx];
344
345             GameObject cellObject = Instantiate(Resources.Load("levelpic")) as GameObject;
346             cellObject.transform.SetParent(chaptercontent, false);
347             card pcard = cellObject.GetComponent<card>();
348             pcard.levelid = lid;
349
350             RectTransform cellRect = cellObject.GetComponent<RectTransform>();
351             // GridLayoutGroup 自动管理位置和尺寸，只需设置 anchor 和 pivot
352             cellRect.anchorMin = new Vector2(0, 0);
353             cellRect.anchorMax = new Vector2(1, 1);
354             cellRect.pivot = new Vector2(0.5f, 0.5f);
355             cellRect.anchoredPosition = Vector2.zero;
```

```
356         cellRect.sizeDelta = Vector2.zero;
357
358         pcard.uvX = (float)i / chapter.ChapterFigureX;
359         pcard.uvY = (float)j / chapter.ChapterFigureY;
360         pcard.uvWidth = 1.0f / chapter.ChapterFigureX;
361         pcard.uvHeight = 1.0f / chapter.ChapterFigureY;
362         pcard.texture = pic;
363         pcard.Load();
364         int showLevel = lid % 100000;
365         pcard.level.text = showLevel.ToString();
366     }
367 }
368
369 btn.gameObject.SetActive(true);
370 levelname.gameObject.SetActive(true);
371 }
372
373 private void ApplyChapterGridLayout(GridLayoutGroup grid, int cols, int rows)
374 {
375     if (grid == null || cols <= 0 || rows <= 0) return;
376
377     Canvas.ForceUpdateCanvases();
378
379     Vector2 rectSize = chaptercontent.rect.size;
380     float availableWidth = Mathf.Max(1f, rectSize.x - grid.padding.left - grid.padding.right);
381     float availableHeight = Mathf.Max(1f, rectSize.y - grid.padding.top - grid.padding.bottom);
382
383     float widthUnits = cols + SpacingXRatio * Mathf.Max(0, cols - 1);
384     float heightUnits = rows + SpacingYRatio * Mathf.Max(0, rows - 1);
385
386     float cardWidthByWidth = availableWidth / widthUnits;
387     float cardHeightByHeight = availableHeight / heightUnits;
388     float cardWidthByHeight = cardHeightByHeight * CardAspect;
389     float cardWidthWithMinSpacing = (availableWidth - MinSpacingX * Mathf.Max(0, cols - 1)) / cols;
390     float cardHeightWithMinSpacing = (availableHeight - MinSpacingY * Mathf.Max(0, rows - 1)) / rows;
391     float cardWidthByMinSpacingHeight = cardHeightWithMinSpacing * CardAspect;
392
393     float cardWidth = Mathf.Min(cardWidthByWidth, cardWidthByHeight, cardWidthWithMinSpacing, cardWidthByMinSpacingHeight ...
394     cardWidth = Mathf.Max(1f, cardWidth);
395     float cardHeight = cardWidth / CardAspect;
396
397     grid.cellSize = new Vector2(cardWidth, cardHeight);
398     grid.spacing = new Vector2(
399         Mathf.Max(MinSpacingX, cardWidth * SpacingXRatio),
400         Mathf.Max(MinSpacingY, cardHeight * SpacingYRatio)
401     );
402     grid.startCorner = GridLayoutGroup.Corner.UpperLeft;
403     grid.startAxis = GridLayoutGroup.Axis.Horizontal;
404     grid.childAlignment = TextAnchor.MiddleCenter;
405     grid.constraint = GridLayoutGroup.Constraint.FixedColumnCount;
```

```
406     grid.constraintCount = cols;
407 }
408
409 private void UpdateStaminaDisplay()
410 {
411     float v = (float)PlayerData.gd.power / 100f;
412     mpower.Value = v;
413 }
414 }

// File: client/Assets/Scripts/ui/frm_game.cs
1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4  using UnityEngine.UI;
5  using TMPro;
6  using System;
7
8  public class frm_game : frmbase
9  {
10     public picmgr mgr;
11     public TextMeshProUGUI level;
12     public Button next;
13     public Button back;
14     public Button setup;
15     private void Awake()
16     {
17         Main.RegistEvent("level_play", (x) =>
18         {
19             if (PlayerData.DebugForcePowerZero)
20             {
21                 PlayerData.gd.power = 0;
22             }
23
24             // 检查并消耗power
25             if (!PlayerData.gd.hasEnoughpower(10))
26             {
27                 Debug.Log("power不足，无法开始游戏");
28                 Main.DispEvent("show_rewarded_power");
29                 return 0;
30             }
31
32             PlayerData.gd.消耗power(10);
33
34             next.gameObject.SetActive(false);
35             level.gameObject.SetActive(false);
36             back.gameObject.SetActive(false);
37             setup.gameObject.SetActive(false);
38
39             mgr.ResizeChapterContent();
```

```
40     var leevel = datamgr.Instance.GetLevel((int)x);
41     level.text = $"当前关卡 {leevel.Id / 100000}-{leevel.Id % 100000}";
42
43     Main.DispEvent("event_loading", true);
44     show();
45     StartCoroutine(load(leevel));
46     return 1;
47 });
48 Main.RegistEvent("level_next", (x) =>
49 {
50     hide();
51     return null;
52 });
53 next.onClick.AddListener(() =>
54 {
55     Main.SendEvent("level_next");
56     hide();
57 });
58 Main.RegistEvent("show_victory", (a) =>
59 {
60     hide();
61     return null;
62 });
63 back.onClick.AddListener(() =>
64 {
65     Main.SendEvent("level_back");
66     hide();
67 });
68 setup.onClick.AddListener(() =>
69 {
70     Main.DispEvent("show_setup");
71 });
72 }
73 IEnumerator load(cfg.DrLevel leevel)
74 {
75     yield return StartCoroutine(mgr.LoadLevel(leevel));
76
77     level.gameObject.SetActive(true);
78     back.gameObject.SetActive(true);
79     setup.gameObject.SetActive(true);
80
81     if (leevel.DifficultyTier == 2)
82     {
83         var df = gb.Find("diff");
84         df.gameObject.SetActive(true);
85         DG.Tweening.DOVirtual.Float(0, 1, .5f, (xvx) =>
86         {
87             df.GetComponent<CanvasGroup>().alpha = xvx;
88         }).onComplete = () =>
89         {
```

```
90         DG.Tweening.DOVirtual.DelayedCall(1, () =>
91         {
92             DG.Tweening.DOVirtual.Float(1, 0, .5f, (xvx) =>
93             {
94                 df.GetComponent<CanvasGroup>().alpha = xvx;
95             }).onComplete = () =>
96             {
97                 df.gameObject.SetActive(false);
98             };
99         });
100     };
101 }
102 }
103
104 // Start is called before the first frame update
105 void Start()
106 {
107
108 }
109
110 // Update is called once per frame
111 void Update()
112 {
113
114 }
115 }
```

```
// File: client/Assets/frm_victory.cs
1 using cfg;
2 using System.Collections;
3 using System.Collections.Generic;
4 using UnityEngine;
5 using UnityEngine.UI;
6
7 public class frm_victory : frmbase
8 {
9     private const float DesignWidth = 720f;
10    private const float DesignHeight = 1280f;
11    private const float DefaultPictureAspect = 16f / 9f;
12
13    public RawImage img;
14    public Button close;
15
16    private RectTransform rootRect;
17    private RectTransform middleGroup;
18    private RectTransform titleGroup;
19    private RectTransform pictureGroup;
20    private RectTransform frameGroup;
21    private RectTransform rewardGroup;
22    private RectTransform bottomGroup;
```

```
23 private RectTransform coinGroup;
24 private RectTransform closeRect;
25 private Vector2 lastRootSize;
26 private float layoutScale = 1f;
27 private float pictureAspect = DefaultPictureAspect;
28 private bool isApplyingLayout;
29
30 private void Awake()
31 {
32     CacheLayoutRefs();
33
34     Main.RegistEvent("show_victory", (x)=>
35     {
36         show();
37         ApplyResponsiveLayout();
38
39         DrLevel leevel = x as DrLevel;
40         Main.inst.StartCoroutine(Main.LoadTextureFromCDN(leevel.LevelFigure, (texture) =>
41         {
42             img.texture = texture;
43             UpdatePictureAspect(texture);
44             ApplyResponsiveLayout();
45         }));
46
47         return null;
48     });
49     close.onClick.AddListener(()=>
50     {
51         Main.SendEvent("level_next");
52         hide();
53     });
54 }
55
56 protected override void OnShow()
57 {
58     CacheLayoutRefs();
59     ApplyResponsiveLayout();
60 }
61
62 private void OnRectTransformDimensionsChange()
63 {
64     if (isActiveAndEnabled && rootRect != null && rootRect.gameObject.activeInHierarchy)
65     {
66         ApplyResponsiveLayout();
67     }
68 }
69
70 // Start is called before the first frame update
71 void Start()
72 {
```



```
73
74     }
75
76     // Update is called once per frame
77     void Update()
78     {
79         if (rootRect == null || !rootRect.gameObject.activeInHierarchy)
80         {
81             return;
82         }
83
84         Vector2 rootSize = GetRootSize();
85         if ((rootSize - lastRootSize).sqrMagnitude > 0.5f)
86         {
87             ApplyResponsiveLayout();
88         }
89     }
90
91     private void CacheLayoutRefs()
92     {
93         rootRect = gb as RectTransform;
94         if (rootRect == null)
95         {
96             return;
97         }
98
99         middleGroup = rootRect.Find("MiddleGroup") as RectTransform;
100        titleGroup = middleGroup != null ? middleGroup.Find("TitleGroup") as RectTransform : null;
101        pictureGroup = middleGroup != null ? middleGroup.Find("PictureGroup") as RectTransform : null;
102        frameGroup = pictureGroup != null ? pictureGroup.Find("FrameGroup") as RectTransform : null;
103        rewardGroup = rootRect.Find("RewardGroup") as RectTransform;
104        bottomGroup = rootRect.Find("BottomGroup") as RectTransform;
105        coinGroup = rootRect.Find("CoinGroup") as RectTransform;
106        closeRect = close != null ? close.GetComponent<RectTransform>() : null;
107    }
108
109    private void ApplyResponsiveLayout()
110    {
111        if (isApplyingLayout)
112        {
113            return;
114        }
115
116        isApplyingLayout = true;
117        try
118        {
119            if (rootRect == null)
120            {
121                CacheLayoutRefs();
122            }
```

```
123
124     if (rootRect == null)
125     {
126         return;
127     }
128
129     Canvas.ForceUpdateCanvases();
130
131     Vector2 rootSize = GetRootSize();
132     lastRootSize = rootSize;
133
134     float sidePadding = Mathf.Clamp(rootSize.x * 0.05f, 24f, 48f);
135     float verticalPadding = Mathf.Clamp(rootSize.y * 0.035f, 24f, 56f);
136     float contentWidth = Mathf.Max(1f, rootSize.x - sidePadding * 2f);
137     float contentHeight = Mathf.Max(1f, rootSize.y - verticalPadding * 2f);
138     layoutScale = Mathf.Min(contentWidth / DesignWidth, contentHeight / DesignHeight);
139     layoutScale = Mathf.Clamp(layoutScale, 0.45f, 1.15f);
140
141     float designContentWidth = DesignWidth * layoutScale;
142     float topY = rootSize.y * 0.5f - verticalPadding;
143     float bottomY = -rootSize.y * 0.5f + verticalPadding;
144     UpdatePictureAspect(img != null ? img.texture : null);
145
146     StretchToParent(middleGroup);
147
148     float titleHeight = 147f * layoutScale;
149     float titleY = topY - titleHeight * 0.5f - 18f * layoutScale;
150     SetCentered(titleGroup, new Vector2(0f, titleY), new Vector2(designContentWidth, titleHeight));
151
152     float bottomHeight = 140f * layoutScale;
153     float bottomCenterY = bottomY + bottomHeight * 0.5f;
154     SetCentered(bottomGroup, new Vector2(0f, bottomCenterY), new Vector2(designContentWidth, bottomHeight));
155
156     float buttonWidth = 320f * layoutScale;
157     float buttonHeight = 108f * layoutScale;
158     SetCentered(closeRect, new Vector2(0f, 12f * layoutScale), new Vector2(buttonWidth, buttonHeight));
159
160     float rewardHeight = 60f * layoutScale;
161     float rewardCenterY = bottomCenterY + bottomHeight * 0.5f + rewardHeight * 0.5f + 18f * layoutScale;
162     SetCentered(rewardGroup, new Vector2(0f, rewardCenterY), new Vector2(designContentWidth * 0.84f, rewardHeight));
163
164     float pictureTop = titleY - titleHeight * 0.5f - 24f * layoutScale;
165     float pictureBottom = rewardCenterY + rewardHeight * 0.5f + 24f * layoutScale;
166     float pictureAvailableWidth = Mathf.Max(1f, designContentWidth - 76f * layoutScale);
167     float pictureAvailableHeight = Mathf.Max(1f, pictureTop - pictureBottom);
168     Vector2 pictureSize = FitSize(pictureAvailableWidth, pictureAvailableHeight, pictureAspect);
169     float pictureCenterY = (pictureTop + pictureBottom) * 0.5f;
170     SetCentered(pictureGroup, new Vector2(0f, pictureCenterY), pictureSize);
171     StretchToParent(frameGroup);
172
```

```
173     if (coinGroup != null)
174     {
175         float coinWidth = 200f * layoutScale;
176         float coinHeight = 64f * layoutScale;
177         float coinX = rootSize.x * 0.5f - sidePadding - coinWidth * 0.5f;
178         float coinY = topY - coinHeight * 0.5f;
179         SetCentered(coinGroup, new Vector2(coinX, coinY), new Vector2(coinWidth, coinHeight));
180     }
181
182     ApplyPictureAspect(img != null ? img.texture : null);
183 }
184 finally
185 {
186     isApplyingLayout = false;
187 }
188 }
189
190 private void ApplyPictureAspect(Texture texture)
191 {
192     UpdatePictureAspect(texture);
193
194     if (img == null)
195     {
196         return;
197     }
198
199     RectTransform imageRect = img.rectTransform;
200     RectTransform parentRect = imageRect.parent as RectTransform;
201     Vector2 parentSize = parentRect != null ? parentRect.rect.size : GetRootSize();
202
203     float insetX = 24f * layoutScale;
204     float insetY = 18f * layoutScale;
205     float availableWidth = Mathf.Max(1f, parentSize.x - insetX * 2f);
206     float availableHeight = Mathf.Max(1f, parentSize.y - insetY * 2f);
207     Vector2 imageSize = FitSize(availableWidth, availableHeight, pictureAspect);
208
209     SetCentered(imageRect, Vector2.zero, imageSize);
210 }
211
212 private void UpdatePictureAspect(Texture texture)
213 {
214     if (texture != null && texture.height > 0)
215     {
216         pictureAspect = Mathf.Max(0.01f, (float)texture.width / texture.height);
217     }
218 }
219
220 private Vector2 GetRootSize()
221 {
222     Vector2 size = rootRect != null ? rootRect.rect.size : Vector2.zero;
```

```
223     if (size.x <= 0f || size.y <= 0f)
224     {
225         size = new Vector2(Screen.width, Screen.height);
226     }
227
228     return size;
229 }
230
231 private static Vector2 FitSize(float availableWidth, float availableHeight, float aspect)
232 {
233     aspect = Mathf.Max(0.01f, aspect);
234     if (availableWidth / availableHeight > aspect)
235     {
236         return new Vector2(availableHeight * aspect, availableHeight);
237     }
238
239     return new Vector2(availableWidth, availableWidth / aspect);
240 }
241
242 private static void StretchToParent(RectTransform rect)
243 {
244     if (rect == null)
245     {
246         return;
247     }
248
249     rect.anchorMin = Vector2.zero;
250     rect.anchorMax = Vector2.one;
251     rect.pivot = new Vector2(0.5f, 0.5f);
252     rect.anchoredPosition = Vector2.zero;
253     rect.sizeDelta = Vector2.zero;
254     rect.localScale = Vector3.one;
255 }
256
257 private static void SetCentered(RectTransform rect, Vector2 anchoredPosition, Vector2 size)
258 {
259     if (rect == null)
260     {
261         return;
262     }
263
264     rect.anchorMin = new Vector2(0.5f, 0.5f);
265     rect.anchorMax = new Vector2(0.5f, 0.5f);
266     rect.pivot = new Vector2(0.5f, 0.5f);
267     rect.anchoredPosition = anchoredPosition;
268     rect.sizeDelta = size;
269     rect.localScale = Vector3.one;
270 }
271 }
```